

Preprocessing and Representation

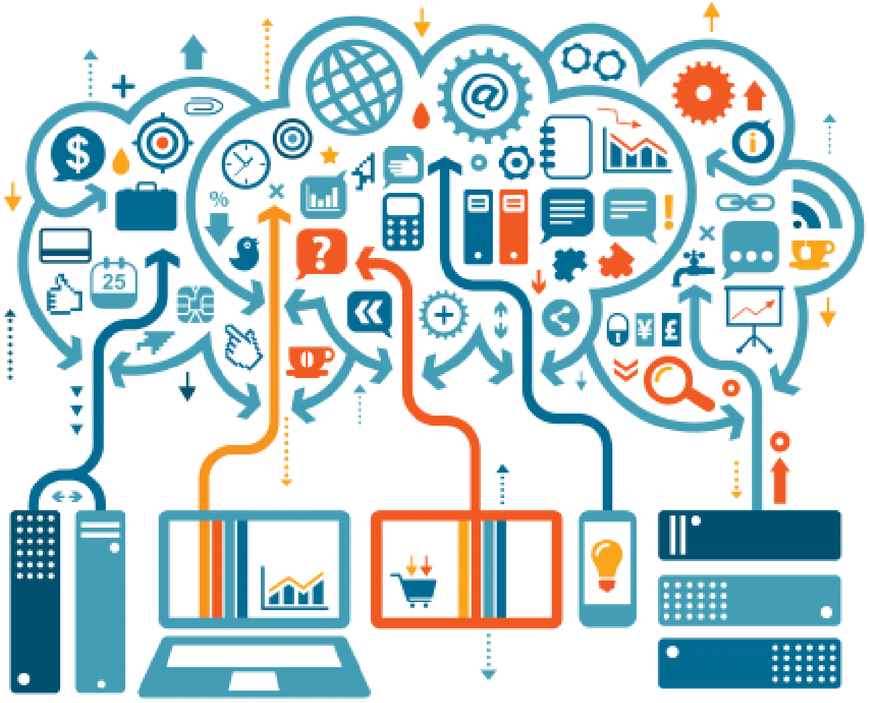
Yung-Chun Chang
Deputy Chief Data Officer, Office of Data Science
Graduate Institute of Data Science, Taipei Medical University, Taiwan.
Email: changyc@tmu.edu.tw

The situation when you process data - Gold pan

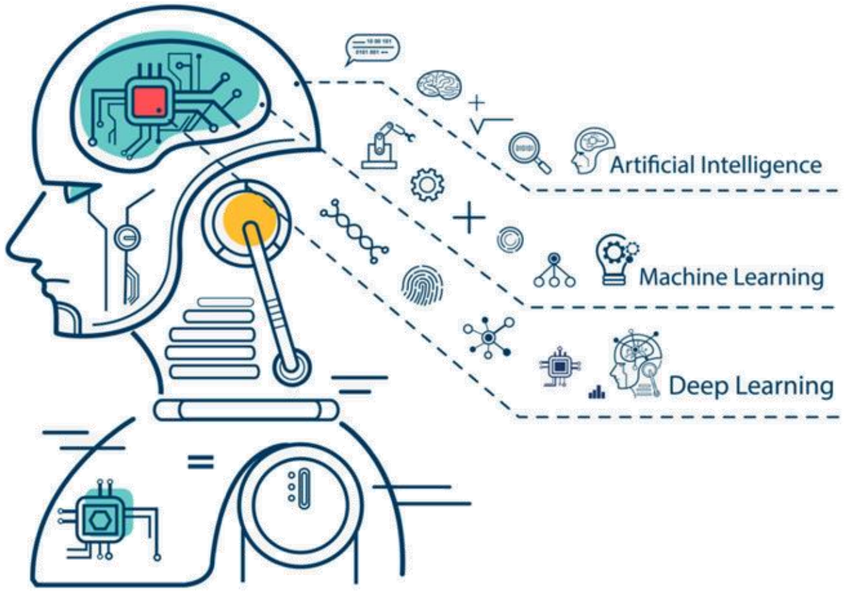


The entire process of artificial intelligence

Data: How to get and store the data?



Action: How to use learned knowledge for action?



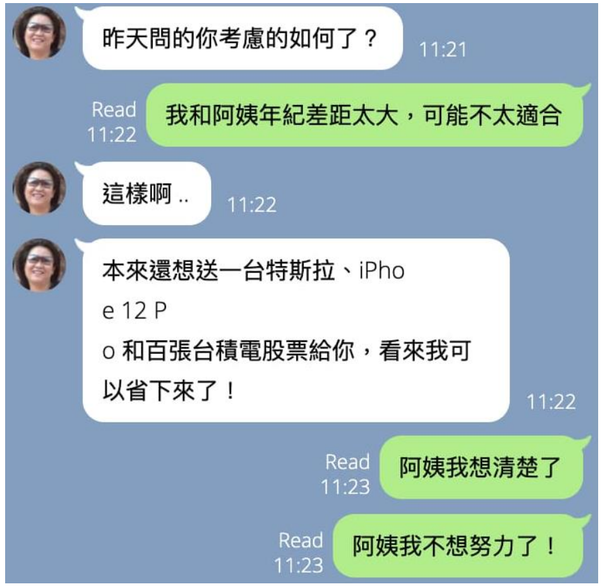
Learning: How to learn knowledge from the data?

What do data looks like?

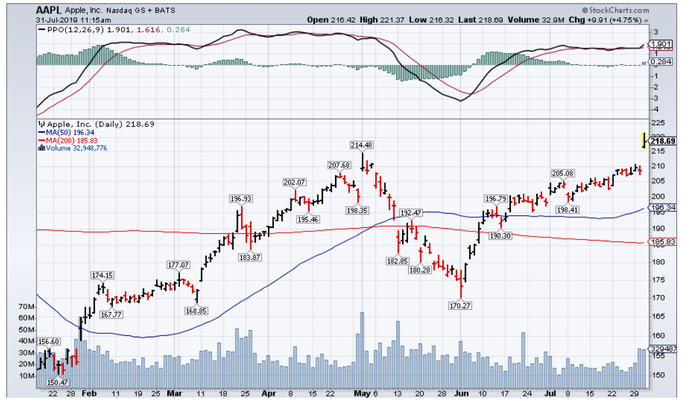
Numerical value

Age, years	42 (35–51)	53 (46–60.75)	63.5 (55.25–73)
Albumin, g/dL	4.6 (4.4–4.8)	4.6 (4.5–4.8)	4.3 (3.8–4.5)
BMI, kg/m²	23.4 (21.1–25.99)	24.7 (22.7–27.02)	25.05 (22.15–27.98)
WC, cm	80.5 (73.5–88)	86 (80–92)	87.58 (82–95.75)
AFP, ng/mL	2.35 (1.62–3.27)	2.62 (1.98–3.62)	2.69 (2.15–4.48)
ALKp, IU/L	60 (50–72)	67 (55–79)	79.69 (65.68–96)
GOT, IU/L	20 (17–24)	23 (20–28)	23.1 (19–34)
GPT, IU/L	18 (13–28)	23 (17–33)	22 (15–35)
T_Bilirubin, mg/dL	0.6 (0.4–0.8)	0.6 (0.5–0.8)	0.6 (0.4–0.9)
γGT, U/L	17 (12–27)	23 (17–34)	34.63 (21–70.90)
CAPscore, dB/m	241 (208–281)	260 (227–301.8)	246 (203–296.5)
Escore, kPa	4.3 (3.5–5.1)	4.6 (3.7–5.6)	7.9 (4.925–13.525)
BUN, mg/dL	12 (10–14.34)	15 (13–18)	23.5 (16.78–38.82)
Creatinine, mg/dL	0.7 (0.6–0.8)	1 (1–1.1)	1.7 (1.425–3.9)
UA, mg/dL	5.1 (4.3–6.3)	6.3 (5.5–7.3)	6.6 (5.7–7.975)
Cholesterol, mg/dL	187 (165–209)	191.5 (165–217)	169 (130.2–195)
HbA1C, %	5.4 (5.2–5.6)	5.525 (5.3–5.9)	5.7 (5.3–6.3)
HDL, mg/dL	54 (45–65)	49 (41–58)	46 (37–57.9)
LDL, mg/dL	121 (102–143)	131 (102.2–152)	107 (80.25–128.75)
TSH, μIU/mL	1.81(1.2–2.545)	2.075 (1.465–2.835)	2.009 (1.54–2.565)

Text



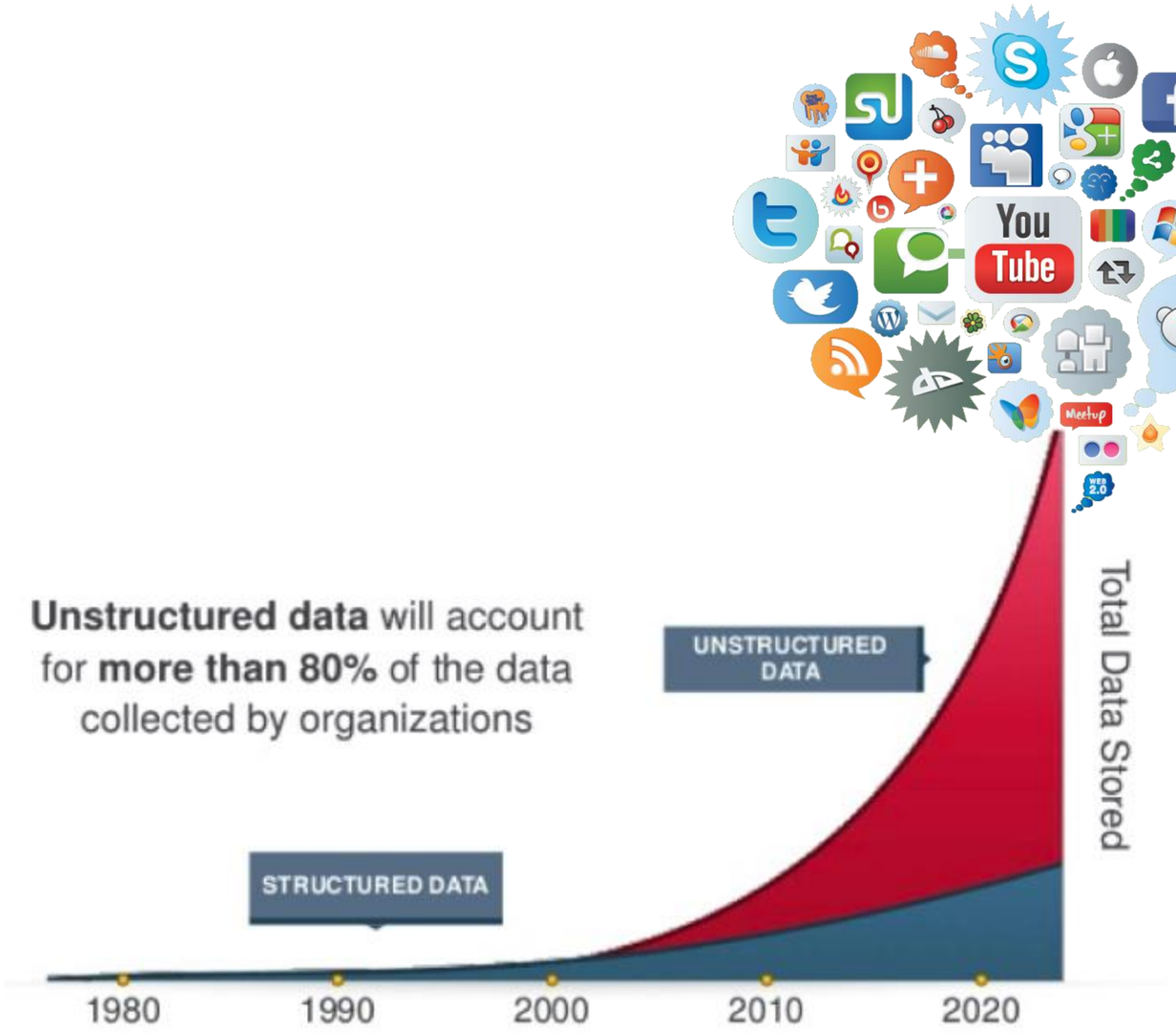
Wave data (dynamic data)



Image



Big Data Growth
is increasingly unstructured



Oracle- Big Data: An Overview



text

image & wave

numerical value

numerical value

text

Mixed data is everywhere

Data Preprocessing

Why we need preprocessing? (1/2)



Why we need preprocessing? (2/2)

- Data in the real world is dirty
  - incomplete: lacking attribute values, lacking certain attributes of interest, or containing only aggregate data
    - e.g., occupation=“”
  - noisy: containing errors or outliers
    - e.g., Salary=“-10”
  - inconsistent: containing discrepancies in codes or names
    - e.g., Age=“42” Birthday=“03/07/1997”
    - e.g., Was rating “1,2,3”, now rating “A, B, C”
    - e.g., discrepancy between duplicate records

Why Is Data Dirty?

- Incomplete data comes from
  - n/a data value when collected
  - different consideration between the time when the data was collected and when it is analyzed.
  - human/hardware/software problems
- Noisy data comes from the process of data
  - collection
  - entry
  - transmission
- Inconsistent data comes from
  - Different data sources
  - Functional dependency violation

Why Is Data Preprocessing Important?

- No quality data, no quality mining results!
  - Quality decisions must be based on quality data
    - e.g., duplicate or missing data may cause incorrect or even misleading statistics.
  - Data warehouse needs consistent integration of quality data
- Data extraction, cleaning, and transformation comprises the majority of the work of building a data warehouse.

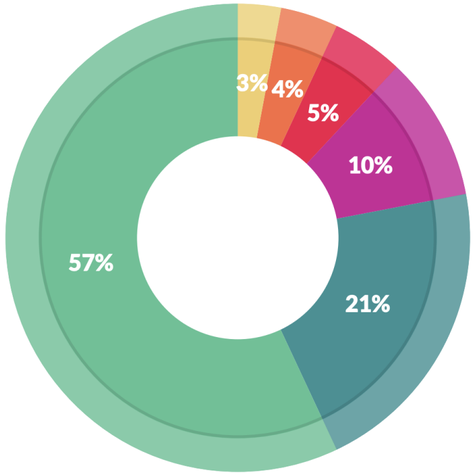


https://www.proaptivity.com/the-cost-of-bad-data/



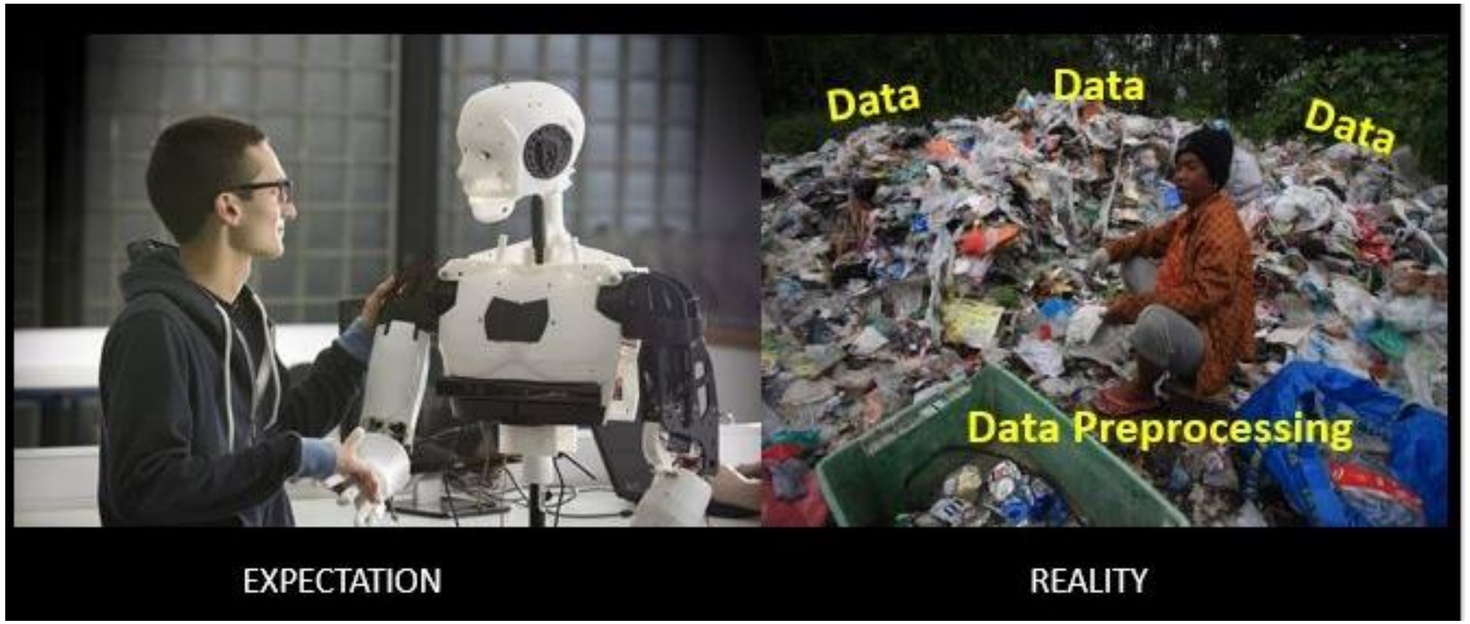
Machine Learning – Expectation VS Reality

Simply put, data wrangling isn't fun. It takes forever. In fact, a few years back, the ***New York Times*** estimated that up to 80% of a data scientist's time is spent doing this sort of work.



What's the least enjoyable part of data science?

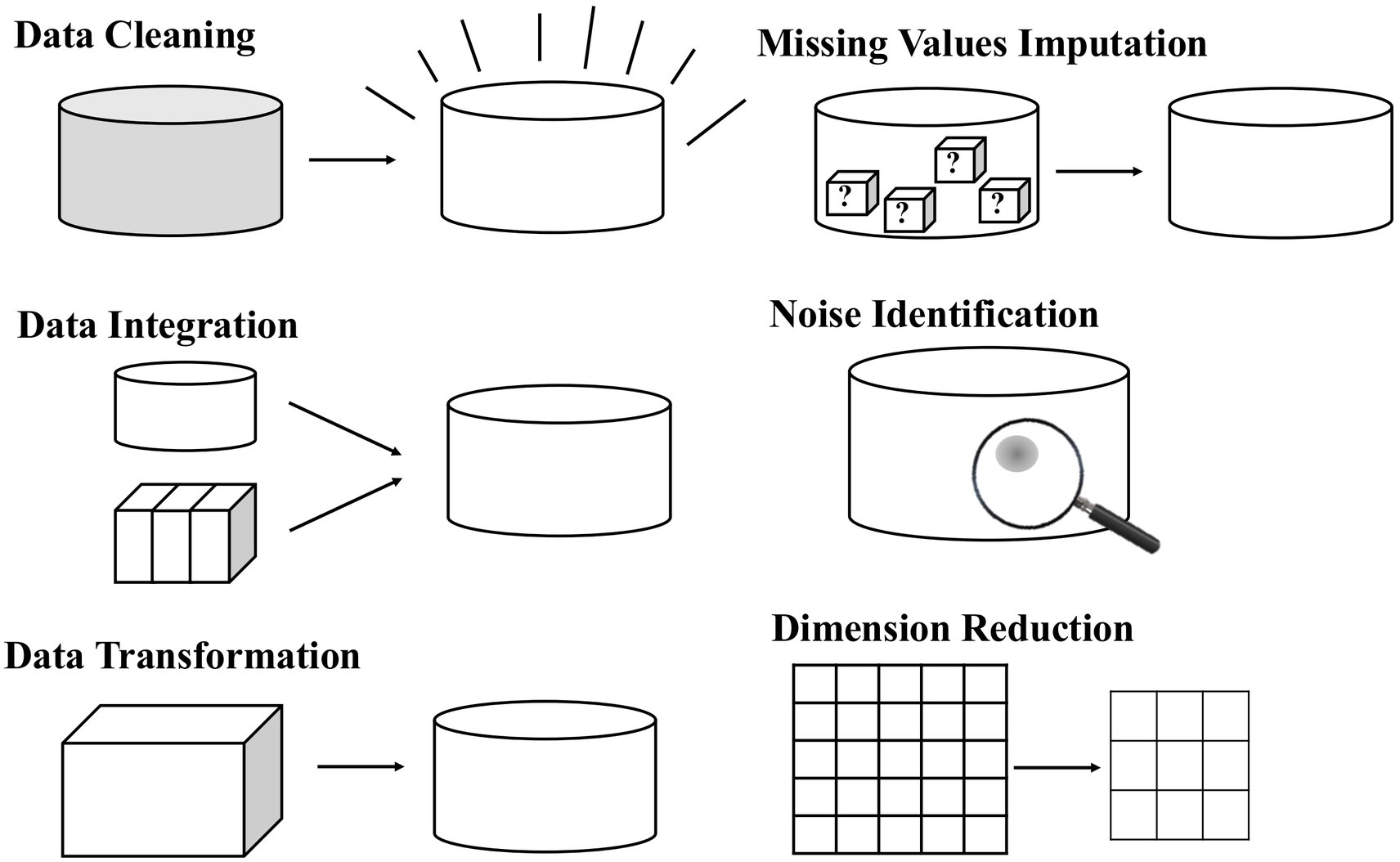
- *Building training sets: 10%*
- *Cleaning and organizing data: 57%*
- *Collecting data sets: 21%*
- *Mining data for patterns: 3%*
- *Refining algorithms: 4%*
- *Other: 5%*



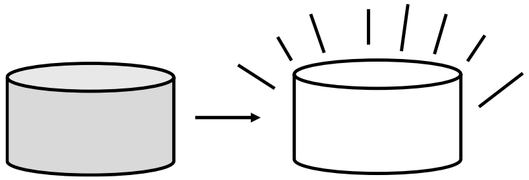
Can AI solve this problem?

https://www.bacancytechnology.com/blog/introduction-to-data-analytics/

Major Tasks in Data Preprocessing



Data Cleaning



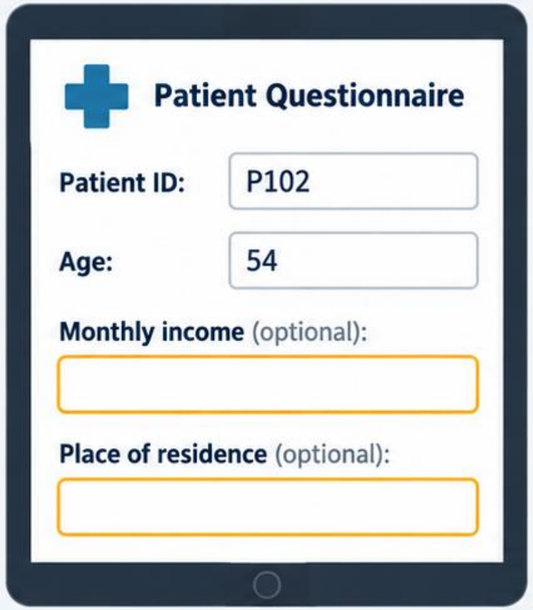
Fill in missing values, smooth noisy data, identify or remove outliers, and resolve inconsistencies

- Fill in the missing value:
  - a global constant : e.g., “unknown”, a new class?!
  - the attribute mean
  - the most probable value: inference-based such as Bayesian formula or decision tree
- Identify outliers and smooth out noisy data
- Correct inconsistent data

Data Cleaning (cont.)

Why Are Values Missing in Healthcare Data?

Patient questionnaire



What happened?



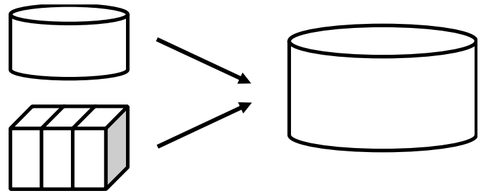
The patient skipped two optional questions.

Recorded dataset

Patient ID	Age	Monthly income	Place of residence
P102	54	Missing	Missing

An unanswered question becomes a missing value in the dataset.

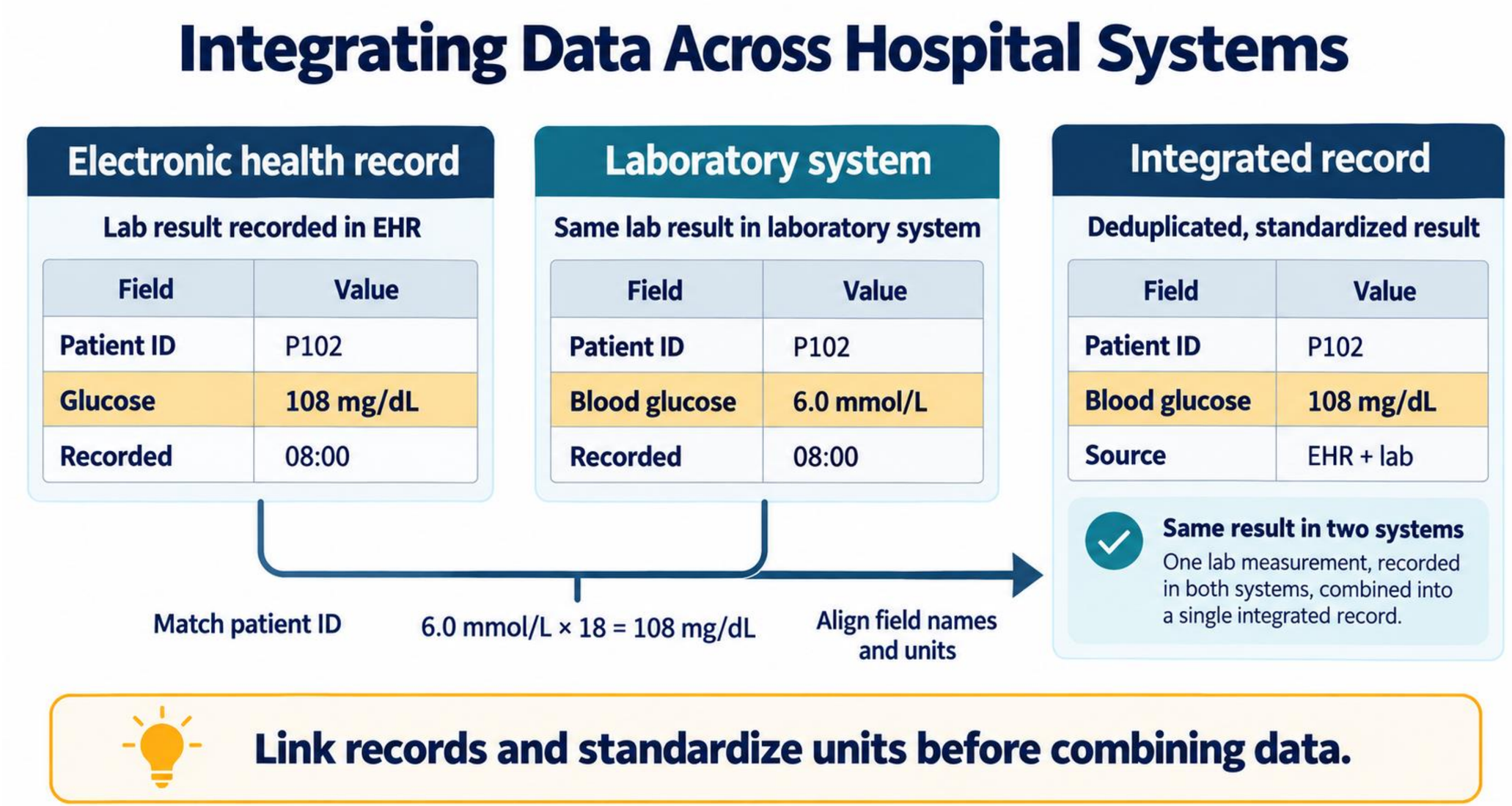
Data Integration



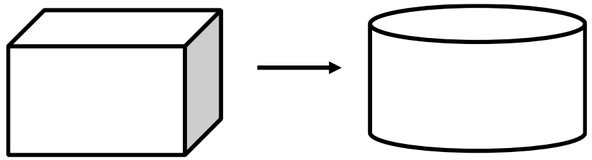
Integration of multiple databases, data cubes, or files

- **Data integration:** combines data from multiple sources into a coherent store
- **Schema integration:** integrate metadata from different sources (e.g. entity identification and linking)
- **Detecting and resolving data value conflicts**
  - for the same real world entity, attribute values from different sources are different
  - possible reasons: different representations, different scales
  - Redundant data occur often when integration of multiple databases. The same attribute may have different names in different databases

Data Integration (cont.)



Data Transformation



Normalization and aggregation

- **Data Transformation**
  - Normalization: scaled to fall within a small, specified range
    - min-max and z-score normalization

$$z_i = \frac{x_i - min(x)}{man(x) - min(x)} \qquad z_i = \frac{x_i - mean(x)}{stand_dev(x)}$$

where $x=(x_1,\ldots,x_n)$ and z_i is now your i^{th} normalized data

Data Transformation (cont.)

Scaling Blood Pressure Data

Original data

Systolic blood pressure (mmHg)

Patient	Systolic BP (mmHg)
A	100
B	**120**
C	140

Min–max scaling
(scaled to [0, 1])

For **Patient B** (120 mmHg):

$$z = \frac{120 - 100}{140 - 100} = \mathbf{0.50}$$



Scaled value (range: 0 to 1)

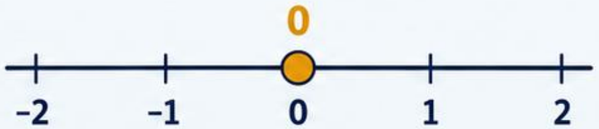
Z-score standardization
(centered at 0, not restricted to [0, 1])

Mean = 120 mmHg

Population SD ≈ 16.33 mmHg

For **Patient B** (120 mmHg):

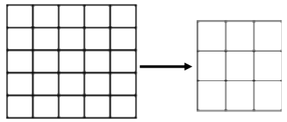
$$z = \frac{120 - 120}{16.33} = \mathbf{0}$$



Standardized value (mean = 0)

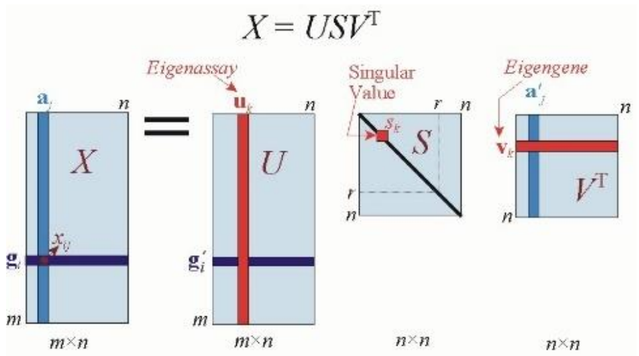
The same blood pressure reading can be expressed on different scales.

Dimension Reduction

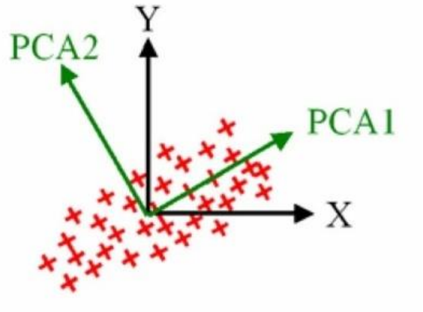


Obtains reduced representation in volume but produces the same or similar analytical results.

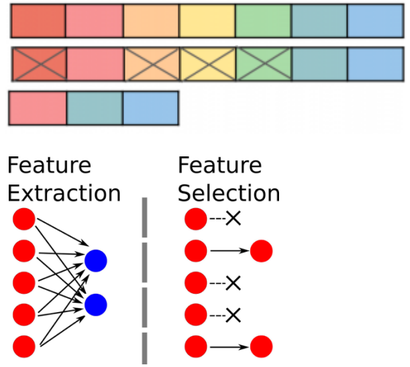
Singular Value Decomposition



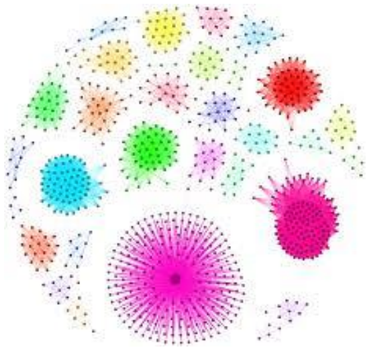
Principal Component Analysis



Feature Selection

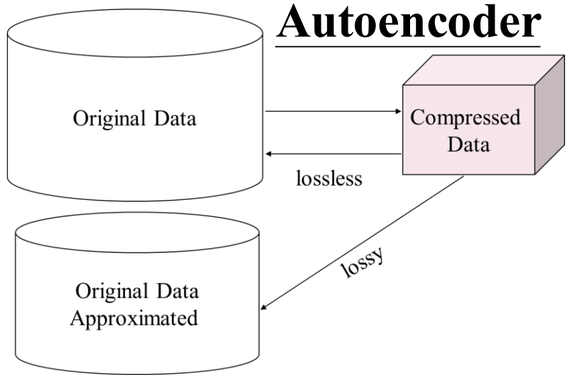


Clustering

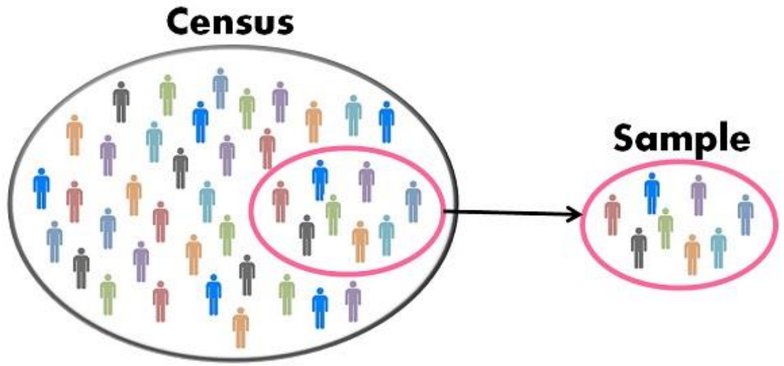


Data Compression

Autoencoder



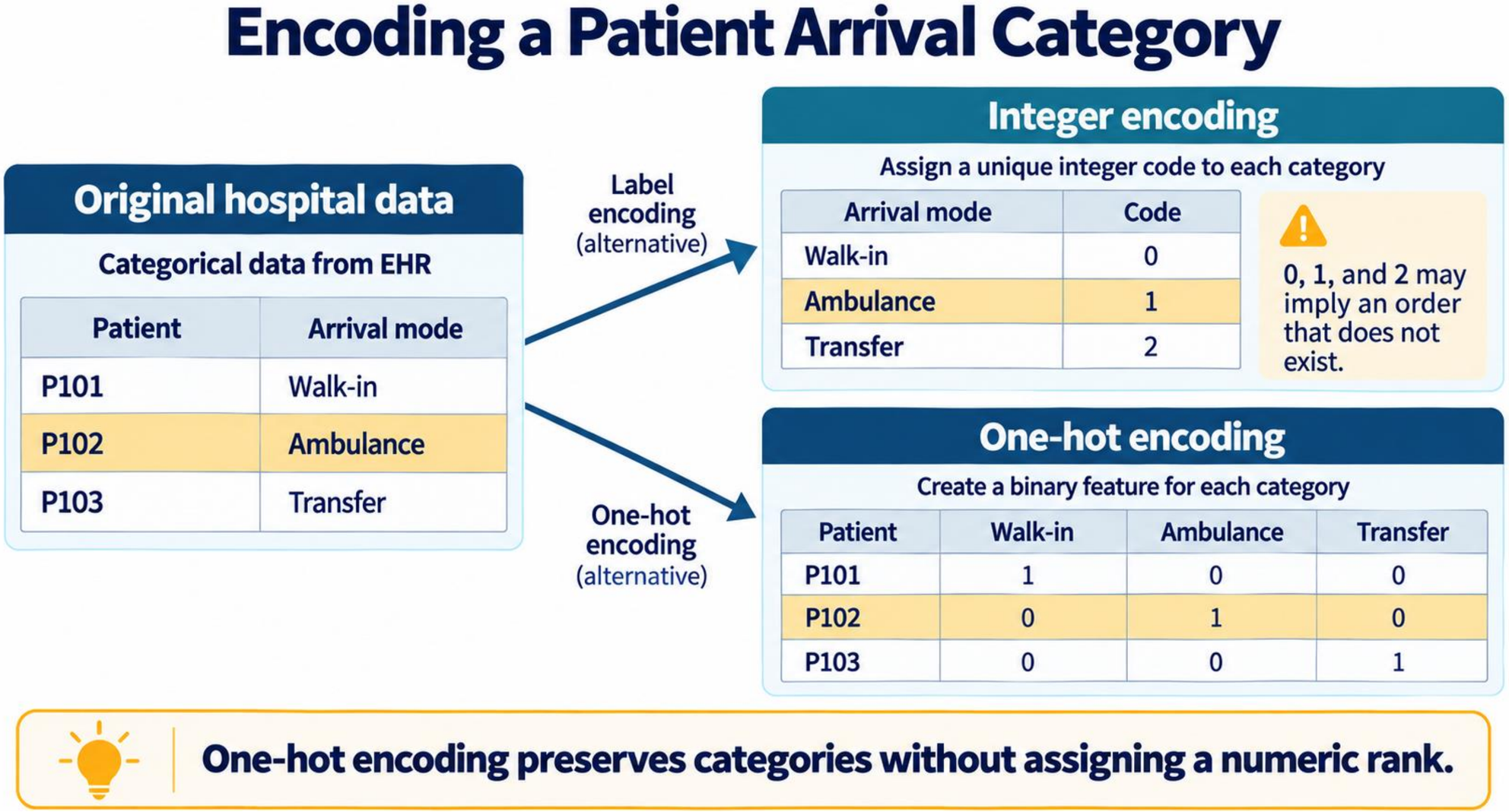
Sampling



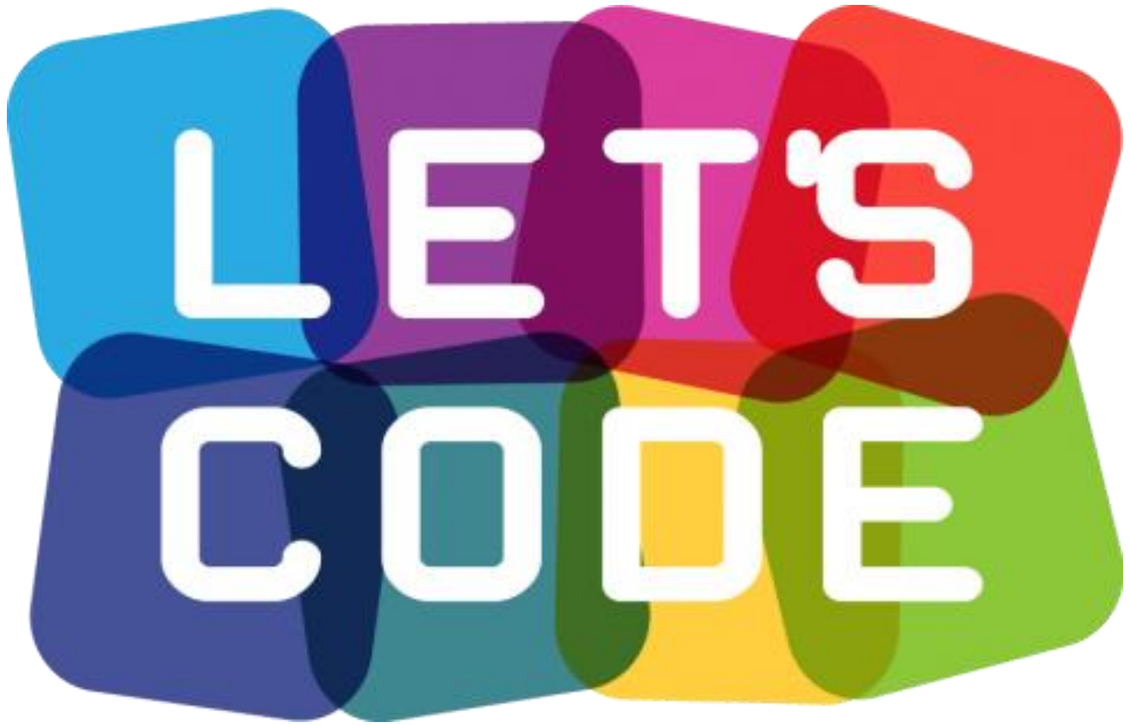
Categorical Encoding
Label encoding V.S. One-Hot Encoding

- Typically, any structured dataset includes multiple columns – a combination of numerical as well as categorical variables. A **machine can only understand the numbers**. It cannot understand the text. That's essentially the case with Machine Learning algorithms too.
- That's primarily the reason we need to convert categorical columns to numerical columns so that a machine learning algorithm understands it. This process is called categorical encoding.
- **Label Encoding** is a popular encoding technique for handling categorical variables. In this technique, each label is assigned a unique integer based on *alphabetical ordering*.
- **One-Hot Encoding** (dummy variables) is another popular technique for treating categorical variables. It simply creates additional features based on the number of unique values in the categorical feature. Every unique value in the category will be added as a feature.

Categorical Encoding
Label encoding V.S. One-Hot Encoding



Data Preprocessing using Titanic Dataset



The Dataset

- https://github.com/jbryer/CompStats/blob/master/Data/titanic3.csv

Download

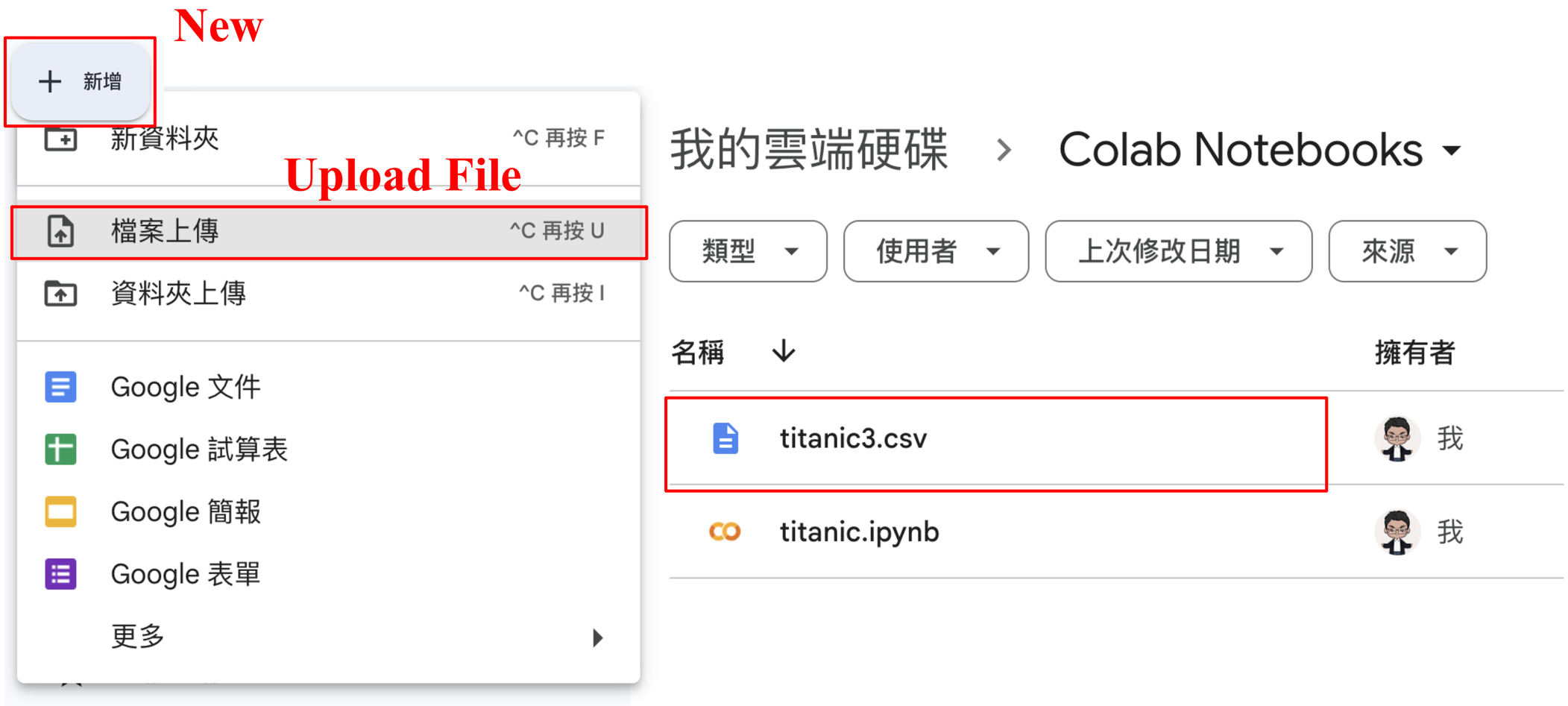
Preview | Code | Blame 1310 lines (1310 loc) · 114 KB Raw

Search this file

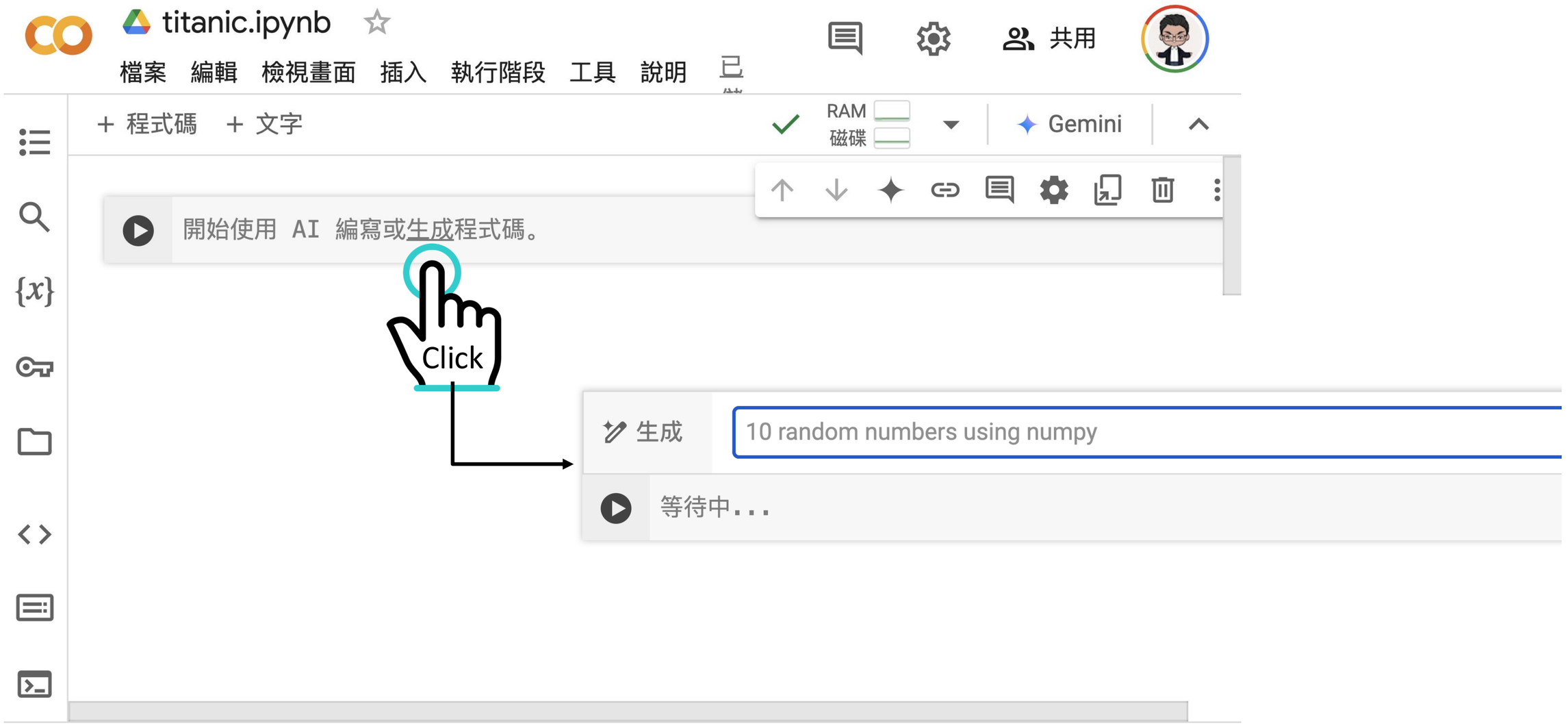
1	pclass	survived	name	sex	age	sibsp	parch	ticket	fare
2	1	1	Allen, Miss. Elisabeth Walton	female	29	0	0	24160	211.3375
3	1	1	Allison, Master. Hudson Trevor	male	0.92	1	2	113781	151.5500
4	1	0	Allison, Miss. Helen Loraine	female	2	1	2	113781	151.5500
5	1	0	Allison, Mr. Hudson Joshua Creighton	male	30	1	2	113781	151.5500
6	1	0	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25	1	2	113781	151.5500
7	1	1	Anderson, Mr. Harry	male	48	0	0	19952	26.5500
8	1	1	Andrews, Miss. Kornelia Theodosia	female	63	1	0	13502	77.9583
9	1	0	Andrews, Mr. Thomas Jr	male	39	0	0	112050	0.0000
10	1	1	Appleton, Mrs. Edward Dale (Charlotte Lamson)	female	53	2	0	11769	51.4792



Upload dataset to Google Drive



Use prompt for coding



Load Dataset

生成

2

我有一個檔名為titanic3.csv的資料集與現在正在撰寫的程式都放在Google Drive 同樣位置，都是放在Colab Notebooks資料夾下，請產生讀取此檔案的程式碼

1

請根據自己的檔案位置改寫
例如: google drive網址

關閉

1 of 1

請謹慎使用程式碼

48 秒

```python
# prompt: 我有一個檔名為titanic3.csv的資料集與現在正在撰寫的程式都放在Google Drive 同樣位置，都是放在Colab Notebooks資料夾下，請產生讀取此檔案的程式碼

import pandas as pd

# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Specify the file path
file_path = '/content/drive/MyDrive/Colab Notebooks/titanic3.csv'

# Read the CSV file into a pandas DataFrame
try:
  df = pd.read_csv(file_path)
  print("File loaded successfully.")
  # Now you can work with the DataFrame 'df'
  # Example: print the first 5 rows
  print(df.head())
except FileNotFoundError:
  print(f"Error: File not found at {file_path}")
except pd.errors.EmptyDataError:
  print(f"Error: The file at {file_path} is empty.")
except pd.errors.ParserError:
  print(f"Error: Could not parse the file at {file_path}. Please check if it's a valid CSV file.")
except Exception as e:
  print(f"An unexpected error occurred: {e}")
```

prompt>_

I have a dataset named `titanic3.csv` and the notebook I am currently writing stored in the **same folder on Google Drive**, specifically under the `Colab Notebooks` directory.

Please generate the code to load this file.

Load Dataset

要允許這個筆記本存取你的 Google 雲端硬碟檔案嗎?

這個筆記本要求存取你的 Google 雲端硬碟檔案。獲得 Google 雲端硬碟存取權後，筆記本中執行的程式碼將可修改 Google 雲端硬碟的檔案。請務必在允許這項存取權前，謹慎審查筆記本中的程式碼。

不用了，謝謝　連線至 Google 雲端硬碟

Allow this notebook access your Google Drive

```
Mounted at /content/drive
File loaded successfully.
   pclass  survived                                             name     sex  \
0       1         1                    Allen, Miss. Elisabeth Walton  female
1       1         1                   Allison, Master. Hudson Trevor    male
2       1         0                     Allison, Miss. Helen Loraine  female
3       1         0             Allison, Mr. Hudson Joshua Creighton    male
4       1         0  Allison, Mrs. Hudson J C (Bessie Waldo Daniels)  female

     age  sibsp  parch  ticket      fare    cabin embarked boat   body  \
0  29.00      0      0   24160  211.3375       B5        S    2    NaN
1   0.92      1      2  113781  151.5500  C22 C26        S   11    NaN
2   2.00      1      2  113781  151.5500  C22 C26        S  NaN    NaN
3  30.00      1      2  113781  151.5500  C22 C26        S  NaN  135.0
4  25.00      1      2  113781  151.5500  C22 C26        S  NaN    NaN

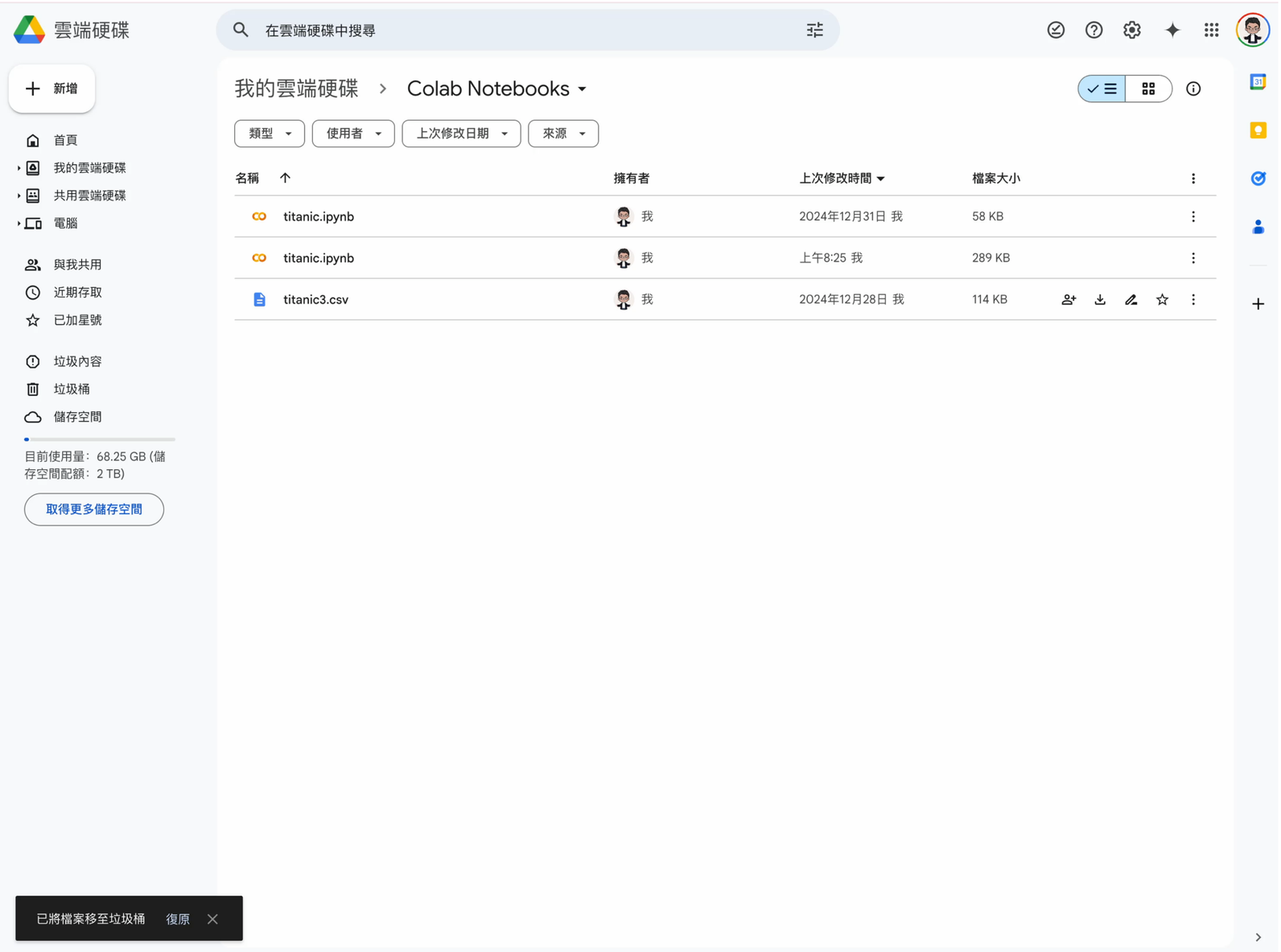
                        home.dest
0                    St Louis, MO
1  Montreal, PQ / Chesterville, ON
2  Montreal, PQ / Chesterville, ON
3  Montreal, PQ / Chesterville, ON
4  Montreal, PQ / Chesterville, ON
```

生成的程式碼可能須遵守授權規範 | MandiraBanerjee2001/Mandira2001

```
df.head(10)
```

	pclass	survived	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked	boat	body	home.dest
0	1	1	Allen, Miss. Elisabeth Walton	female	29.00	0	0	24160	211.3375	B5	S	2	NaN	St Louis, MO
1	1	1	Allison, Master. Hudson Trevor	male	0.92	1	2	113781	151.5500	C22 C26	S	11	NaN	Montreal, PQ / Chesterville, ON
2	1	0	Allison, Miss. Helen Loraine	female	2.00	1	2	113781	151.5500	C22 C26	S	NaN	NaN	Montreal, PQ / Chesterville, ON
3	1	0	Allison, Mr. Hudson Joshua Creighton	male	30.00	1	2	113781	151.5500	C22 C26	S	NaN	135.0	Montreal, PQ / Chesterville, ON

Load Dataset



prompt>_

I have a dataset named titanic3.csv and the notebook I am currently writing stored in the same folder on Google Drive, specifically under the Colab Notebooks directory.
Please generate the code to load this file.

（我有一個檔名為titanic3.csv的資料集與現在正在撰寫的程式都放在Google Drive同樣位置，都是放在Colab Notebooks資料夾下，請產生讀取此檔案的程式碼）

Explore Data and Preprocessing

生成 | survived欄位是存活資訊，1表示存活，0表示死亡，請以這個欄位作為Label (Y)，剩下的作為Feature (X)，請幫我產生兩個變數儲存

1 of 1 | 請謹慎使用程式碼

```
# prompt: survived欄位是存活資訊，1表示存活，0表示死亡，請以這個欄位作為Label (Y)，剩下的作為Feature (X)，請幫我產生兩個變數儲存

# Define features (X) and target (y)
y = df['survived']
X = df.drop('survived', axis=1)
```

prompt>_

The `survived` column represents survival information, where **1 means survived and 0 means did not survive**.

Please use this column as the **label (Y)** and treat all remaining columns as **features (X)**.

Help me create two variables to store them.

生成 | 請以視覺化方式來產生Y的分佈圖

1 of 1 | 請謹慎使用程式碼

```
# prompt: 請以視覺化方式來產生Y的分佈圖

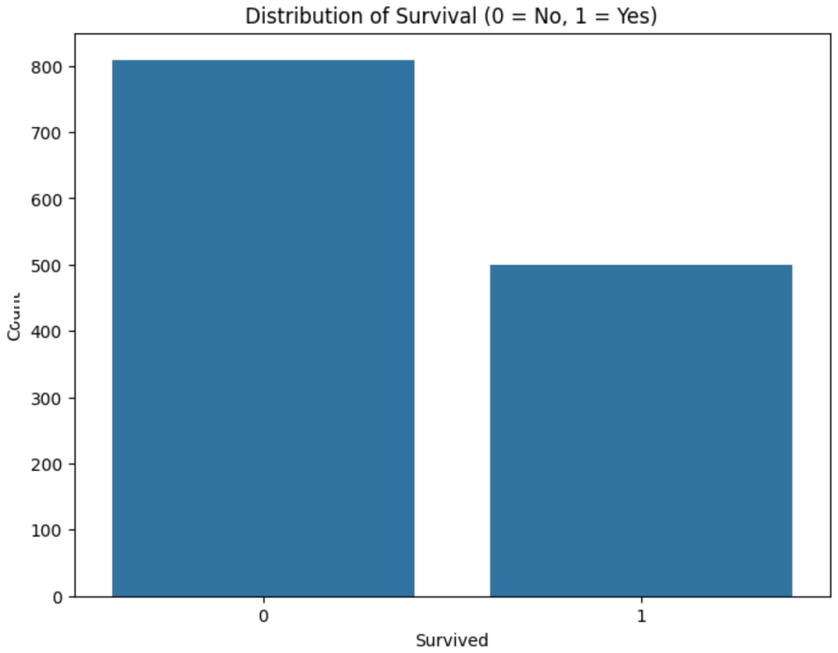
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming 'y' is defined as in the previous code (df['survived'])

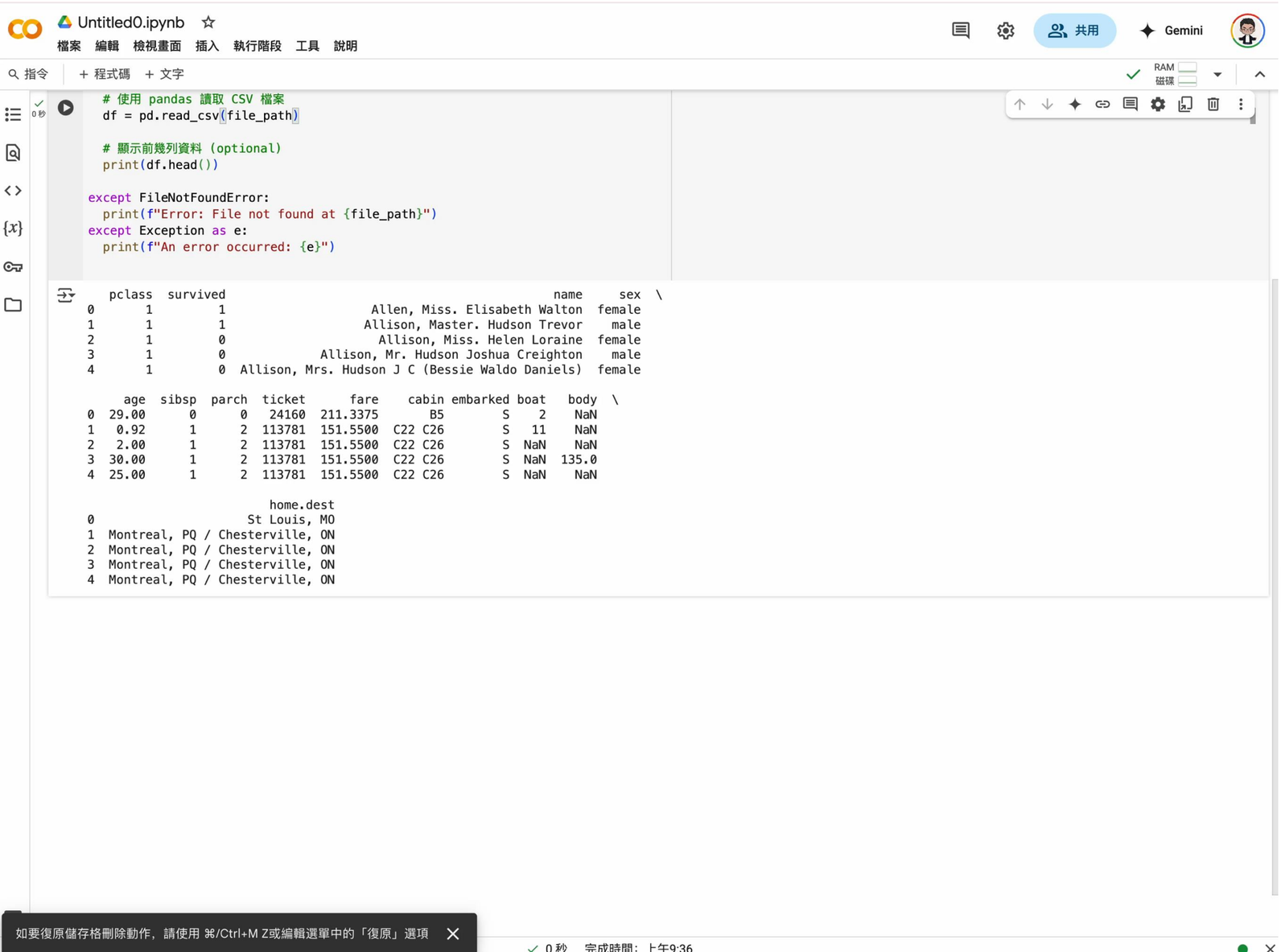
plt.figure(figsize=(8, 6))
sns.countplot(x=y)
plt.title('Distribution of Survival (0 = No, 1 = Yes)')
plt.xlabel('Survived')
plt.ylabel('Count')
plt.show()
```

English Prompt

Please **visualize the distribution of Y** (the `survived` column).



Explore Data and Preprocessing



prompt>_

The survived column represents survival information, where 1 means survived and 0 means did not survive. Please use this column as the label (Y) and treat all remaining columns as features (X). Help me create two variables to store them.

（survived欄位是存活資訊，1表示存活，0表示死亡，請以這個欄位作為Label (Y)，剩下的作為Feature (X)，請幫我產生兩個變數儲存）

prompt>_

Please visualize the distribution of Y (the survived column).

（請以視覺化方式來產生Y的分佈圖）

Explore Data and Preprocessing

生成 | 我想要檢查X中有哪些欄位有遺失值

1 of 1 | 請謹慎使用程式碼

```
# prompt: 我想要檢查X中有哪些欄位有遺失值

# Check for missing values in each column of X
missing_values = X.isnull().sum()
missing_values
```

	0
pclass	0
name	0
sex	0
age	263
sibsp	0
parch	0
ticket	0
fare	1
cabin	1014
embarked	2
boat	823
body	1188
home.dest	564

prompt>_

I would like to check which columns in **X** contain missing values.

- "Age" and "Fare" are numerical data, filled using the **mean**.
- "Embarked" is categorical data, filled using the **mode**.
- Other entries with too many missing values are **removed**.
- "Name", "Ticket", and "Cabin" are irrelevant in predicting survival, so we **exclude** them.



Explore Data and Preprocessing

生成

幫我重新整理X，我不需要"name", "ticket", "cabin", "boat", "body", "home.dest"欄位，針對"age"以及"fare"欄位使用他們各自的平均值來填補缺失值，"embarked"採用眾數來填補缺失值，最後輸出新的X各欄位缺失值讓我檢視

關閉

1 of 1 復原變更 請謹慎使用程式碼

prompt>_

```python
# prompt: 幫我重新整理X，我不需要"name", "ticket", "cabin", "boat", "body",
# "home.dest"欄位，針對"age"以及"fare"欄位使用他們各自的平均值來填補缺失值，"embarked"採用眾數來填補缺失值，
# 最後輸出新的X各欄位缺失值讓我檢視

# Drop specified columns
X = X.drop(['name', 'ticket', 'cabin', 'boat', 'body', 'home.dest'], axis=1)

# Fill missing 'age' values with the mean
X['age'] = X['age'].fillna(X['age'].mean())

# Fill missing 'fare' values with the mean
X['fare'] = X['fare'].fillna(X['fare'].mean())

# Fill missing 'embarked' values with the mode
X['embarked'] = X['embarked'].fillna(X['embarked'].mode()[0])

# Display missing values in each column of the modified X
print(X.isnull().sum())
```

```
pclass      0
sex         0
age         0
sibsp       0
parch       0
fare        0
embarked    0
dtype: int64
```

Please clean **X** as follows:

1. **Remove** the columns:
   `"name"`, `"ticket"`, `"cabin"`, `"boat"`, `"body"`, `"home.dest"`.
2. For the `"age"` and `"fare"` columns, **fill missing values with their respective mean**.
3. For the `"embarked"` column, **fill missing values with the mode**.
4. Finally, **output the missing-value counts of each column in the updated X** so I can review them.

```python
X.head(10)
```

	pclass	sex	age	sibsp	parch	fare	embarked
0	1	female	29.00	0	0	211.3375	S
1	1	male	0.92	1	2	151.5500	S
2	1	female	2.00	1	2	151.5500	S
3	1	male	30.00	1	2	151.5500	S
4	1	female	25.00	1	2	151.5500	S
5	1	male	48.00	0	0	26.5500	S
6	1	female	63.00	1	0	77.9583	S
7	1	male	39.00	0	0	0.0000	S
8	1	female	53.00	2	0	51.4792	S
9	1	male	71.00	0	0	49.5042	C

Explore Data and Preprocessing

Untitled0.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

共用 Gemini

指令 ＋程式碼 ＋文字

RAM 磁碟

```
# prompt: 請以視覺化方式來產生Y的分佈圖

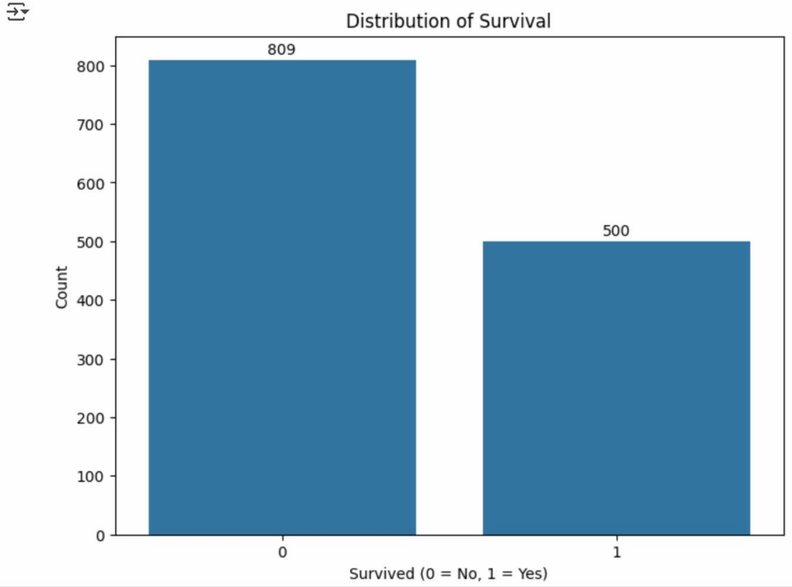
import matplotlib.pyplot as plt
import seaborn as sns

# 計算 y 的值出現的次數
y_counts = y.value_counts()

# 使用 seaborn 的 countplot 繪製 y 的分佈圖
plt.figure(figsize=(8, 6))  # 設定圖表大小
sns.countplot(x=y)
plt.title('Distribution of Survival')
plt.xlabel('Survived (0 = No, 1 = Yes)')
plt.ylabel('Count')

# 顯示每個值的數量
for i, count in enumerate(y_counts):
    plt.text(i, count + 5, str(count), ha='center', va='bottom')

plt.show()
```



0 秒 完成時間：上午9:43

prompt>_

I would like to check which columns in **X** contain missing values.
（我想要檢查X中有哪些欄位有遺失值）

prompt>_

Please clean X as follows:
1. Remove the columns:
"name", "ticket", "cabin", "boat", "body", "home.dest".
2. For the "age" and "fare" columns, fill missing values with their respective mean.
3. For the "embarked" column, fill missing values with the mode.
4. Finally, output the missing-value counts of each column in the updated X so I can review them.
（幫我重新整理X，我不需要“name”, “ticket”, “cabin”, “boat”, “body”, “home.dest”欄位，針對“age”以及“fare”欄位使用他們各自的平均值來填補缺失值，“embarked”採用眾數來填補缺失值，最後輸出新的X各欄位缺失值讓我檢視）

Explore Data and Preprocessing

生成 | 進一步將X中，embarked欄位轉換成One-hot encoding表示，將sex欄位的female轉為0, male為1

1 of 1 | 復原變更 | 請謹慎使用程式碼

```
# prompt: 進一步將X中，embarked欄位轉換成One-hot encoding表示，將sex欄位的female轉為0, male為1

# Convert 'sex' column: female to 0, male to 1
X['sex'] = X['sex'].map({'female': 0, 'male': 1})

# Perform one-hot encoding on 'embarked' column
embarked_dummies = pd.get_dummies(X['embarked'], prefix='embarked').astype(int)

# Concatenate the one-hot encoded columns with the original DataFrame
X = pd.concat([X, embarked_dummies], axis=1)
X = X.drop('embarked', axis=1)

X.head(10)
```

	pclass	sex	age	sibsp	parch	fare	embarked_C	embarked_Q	embarked_S
0	1	0	29.00	0	0	211.3375	0	0	1
1	1	1	0.92	1	2	151.5500	0	0	1
2	1	0	2.00	1	2	151.5500	0	0	1
3	1	1	30.00	1	2	151.5500	0	0	1
4	1	0	25.00	1	2	151.5500	0	0	1
5	1	1	48.00	0	0	26.5500	0	0	1

prompt>_

Next, please update **X** as follows:

1. Convert the `embarked` column into **one-hot encoded features**.
2. Convert the `sex` column so that **female = 0** and **male = 1**.

Explore Data and Preprocessing

生成 | 對age以及fare欄位作MinMax標準化

1 of 1 | 復原變更 | 請謹慎使用程式碼

prompt>_

Please apply **Min-Max normalization** to the `age` and `fare` columns.

```
# prompt: 對age以及fare欄位作MinMax標準化

# ... (Your existing code)

from sklearn.preprocessing import MinMaxScaler

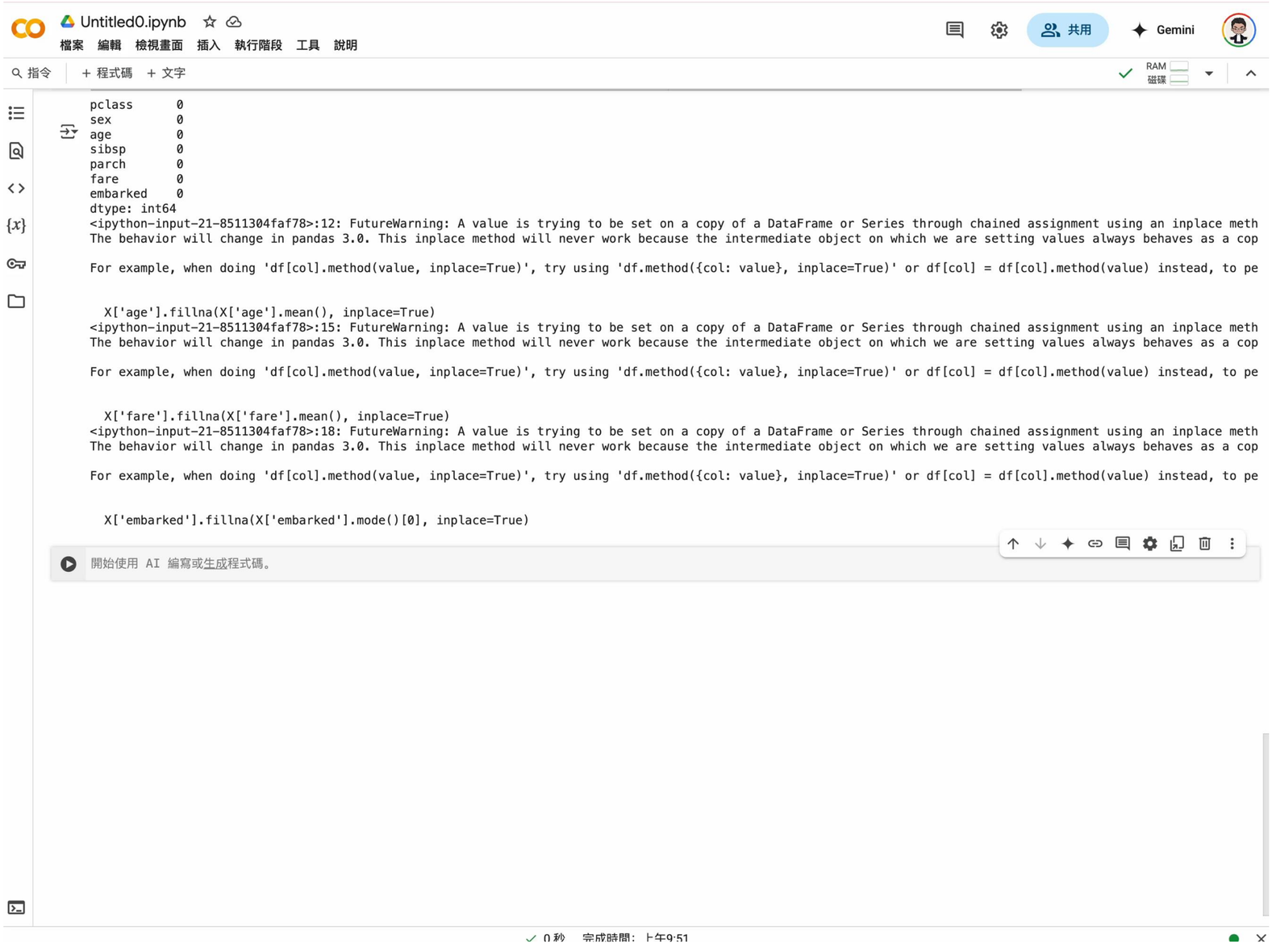
# Create a MinMaxScaler object
scaler = MinMaxScaler()

# Fit and transform the 'age' and 'fare' columns
X[['age', 'fare']] = scaler.fit_transform(X[['age', 'fare']])

X.head(5)
```

	pclass	sex	age	sibsp	parch	fare	embarked_C	embarked_Q	embarked_S
0	1	0	0.361142	0	0	0.412503	0	0	1
1	1	1	0.009395	1	2	0.295806	0	0	1
2	1	0	0.022924	1	2	0.295806	0	0	1
3	1	1	0.373669	1	2	0.295806	0	0	1
4	1	0	0.311036	1	2	0.295806	0	0	1

Explore Data and Preprocessing



prompt>_

Please update X as follows:
1. Convert the embarked column into one-hot encoded features.
2. Convert the sex column so that female = 0 and male = 1.
(進一步將X中，embarked欄位轉換成One-hot encoding表示，將sex欄位的female轉為0, male為1)

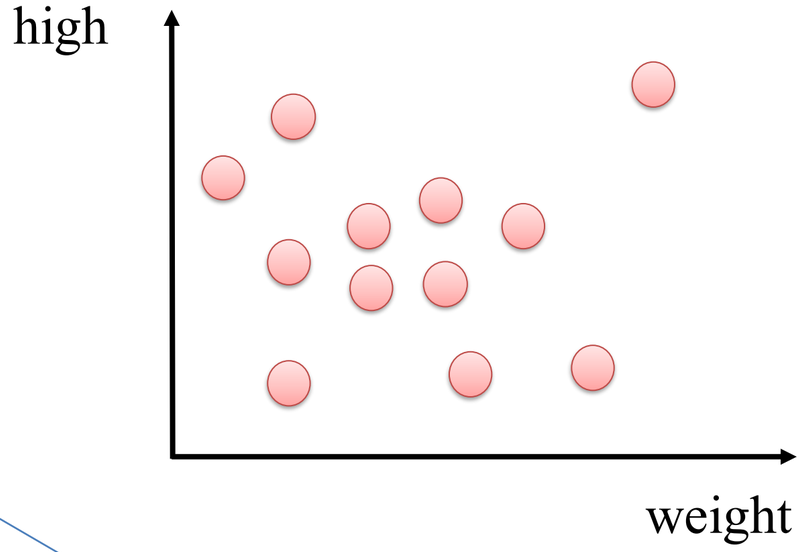
prompt>_

Please apply Min-Max normalization to the age and fare columns.
(對age以及fare欄位作MinMax標準化)

Data Representation

How to represent each people?





160	55	F
High	Weight	Label

178	80	M
High	Weight	Label

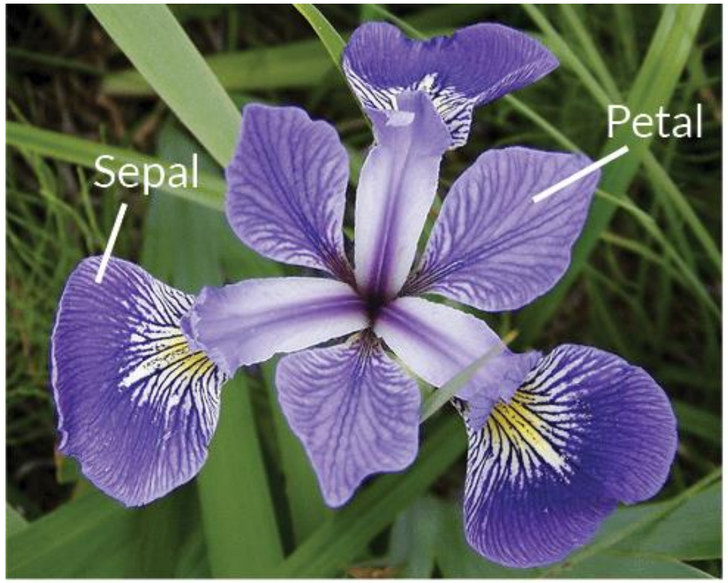
165	60	F
High	Weight	Label

182	82	M
High	Weight	Label

158	48	F
High	Weight	Label

features

How to represent different types of flowers?



Iris Versicolor



Iris Setosa



Iris Virginica

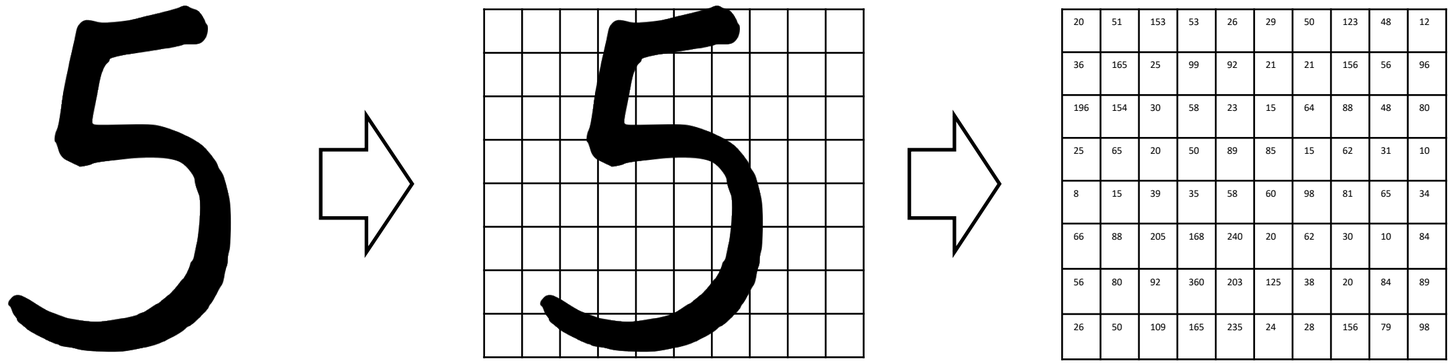
6.4	3.5	4.5	1.2	Versicolor
Sepal length	Sepal width	Petal length	Petal width	Label

5.1	3.5	1.4	0.2	Setosa
Sepal length	Sepal width	Petal length	Petal width	Label

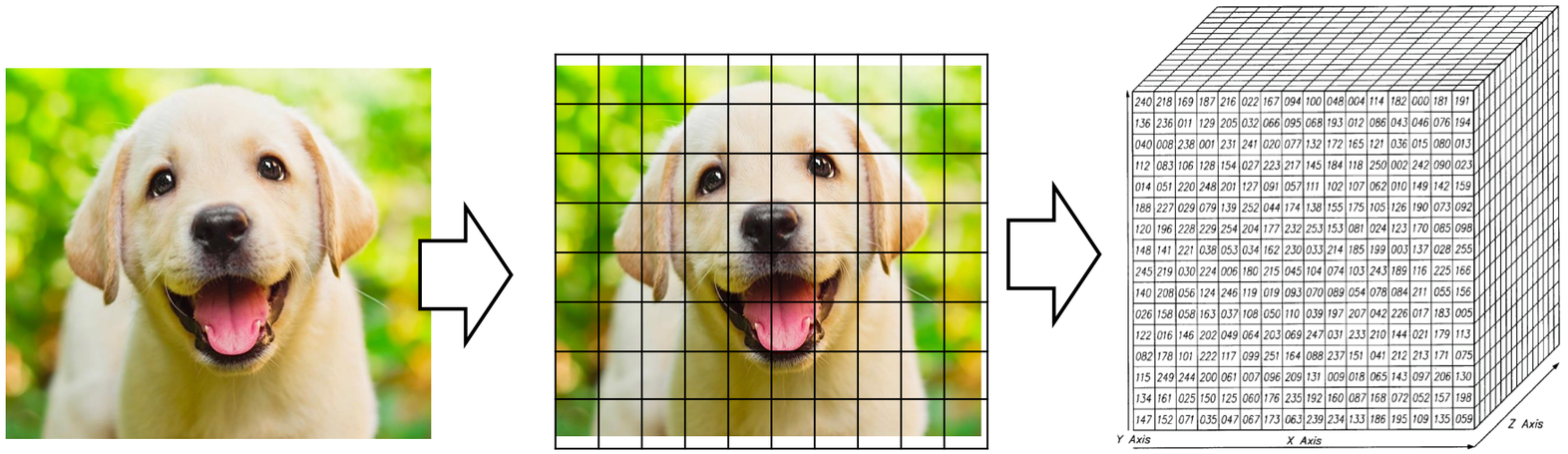
5.9	3.0	5.0	1.8	Virginica
Sepal length	Sepal width	Petal length	Petal width	Label

features

How to represent different types of images?



20	51	153	53	26	29	50	123	48	12
36	165	25	99	92	21	21	156	56	96
196	154	30	58	23	15	64	88	48	80
25	65	20	50	89	85	15	62	31	10
8	15	39	35	58	60	98	81	65	34
66	88	205	168	240	20	62	30	10	84
56	80	92	360	203	125	38	20	84	89
26	50	109	165	235	24	28	156	79	98



How to represent different types of documents?



Book leak: Bolton says Trump tied Ukraine funds to political investigations. A11

DAILY NEWS

SERVING THE SAN FERNANDO AND SURROUNDING VALLEYS

AN EDITION OF THE REGISTER

Monday, January 27, 2020 $2.00 FACEBOOK.COM/LADAILYNEWS TWITTER.COM/LADAILYNEWS dailynews.com

KOBE BRYANT 1978-2020

A LAKER LEGEND

SHOCK, SADNESS AFTER FATAL CRASH

GETTY IMAGES

Sports icon: He grew to greatness as L.A. watched; his ruthless drive, work ethic took team to 5 NBA titles

By Kyle Goon
kgoon@scng.com

From his fans, he inspired devotion. From rivals, frustration and grudging respect. Among the young athletes who admired him, he spawned countless imitators.

The world was stunned Sunday by the news that Kobe Bryant, one of the greatest basketball players of all time whose cultural relevance transcended the court, had died in a helicopter crash at the age of 41 along with his 13-year-old daughter, Gianna, and seven other people.

Throughout a 20-year NBA career and beyond, Bryant's on-court exploits and charisma grew beyond the realm of sport into popular culture. While he was

BRYANT » PAGE 7

MINDY SCHAUER — STAFF PHOTOGRAPHER

Thousands of Kobe Bryant fans converge outside Staples Center to honor the former Laker, who died Sunday in a helicopter crash in Calabasas.

Inside: More coverage of the death of Lakers legend Kobe Bryant. PAGES A5-7, SPORTS

Calabasas: Former NBA star and 13-year-old daughter among nine killed as helicopter hits foggy hillside

By Tony Saavedra, Josh Cain, Eric Licas and Scott Schwebke
Staff writers

CALABASAS » Lakers legend Kobe Bryant and his 13-year-old daughter, Gianna, were among nine people killed Sunday when a helicopter taking them to his youth basketball academy crashed in the foggy hills of Calabasas.

There were no survivors.

Those on board the 12-seat helicopter included Orange Coast College head baseball coach John Altobelli, his wife, Keri, and daughter Alyssa, according to news reports. Another victim was Christina Mauser, who was a coach at Harbor Day School in Corona del Mar. Costa Mesa Mayor Katrina Foley tweeted.

CRASH » PAGE 7

PUBLIC HEALTH

Coronavirus confirmed in L.A., OC

By Richard K. De Atley
rdeatley@scng.com
@RKDeAtley on Twitter

Two cases of coronavirus were confirmed in Southern California, one each in Los Angeles and Orange counties, bringing to five the number of cases across the U.S., health officials said Sunday.

All the patients had traveled to the Chinese city of Wuhan, identified as the center of the outbreak. The U.S. patients generally have been reported to be in good condition and were hospitalized in isolation for monitoring.

The Orange County patient, identified as a man in his 50s, was in good condition Sunday while being treated at an undisclosed hospital, said Dr. Nichole Quick, Orange County Health officer. She said he was "a traveler from Wuhan who is currently staying in Orange County."

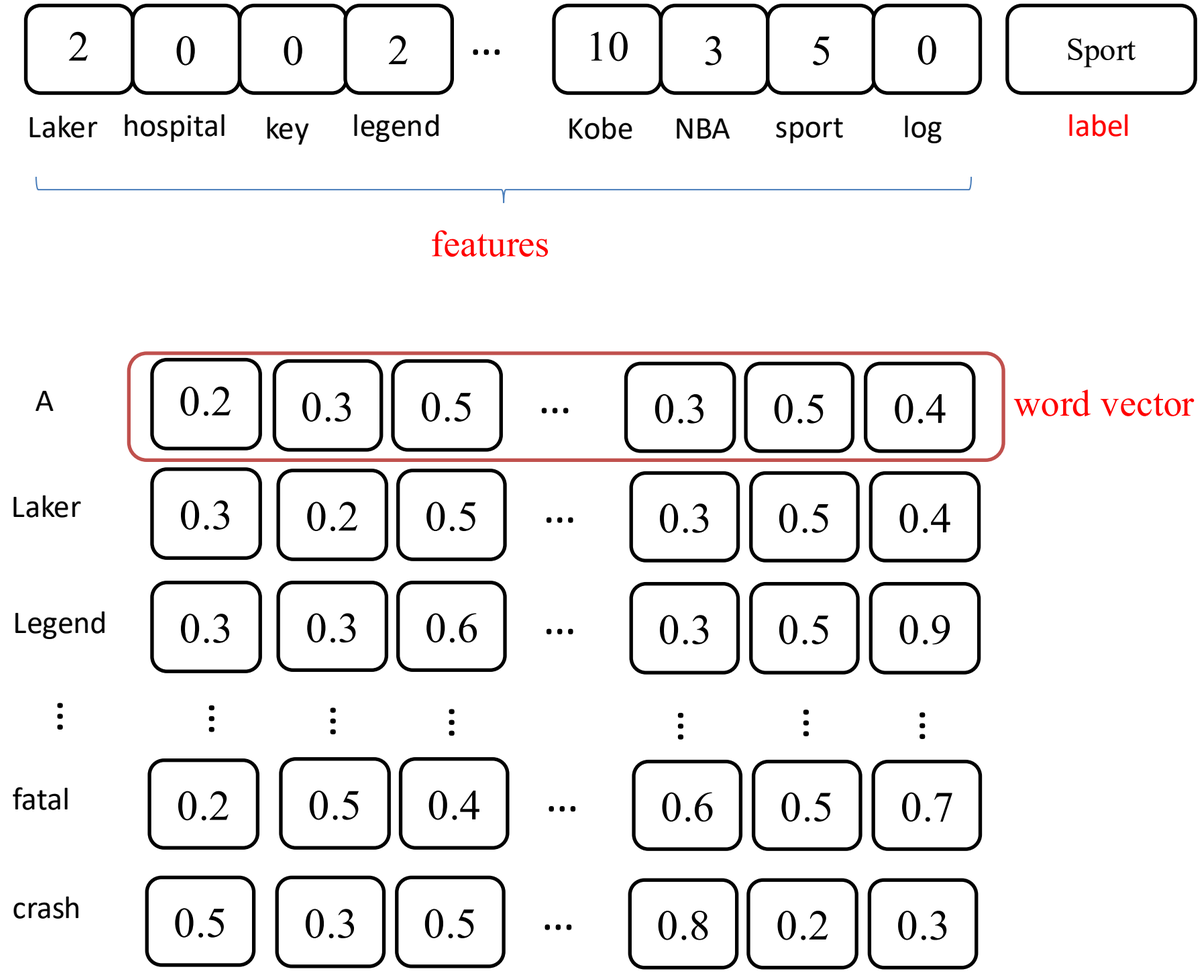
The Orange County Health Care Agency, along with the U.S. Centers for Disease Control

VIRUS » PAGE 4

62ND ANNUAL GRAMMY AWARDS

Alicia Keys, left, and Dua Lipa present Billie Eilish with the award for best new artist at the 62nd annual Grammy Awards on Sunday in Los Angeles.

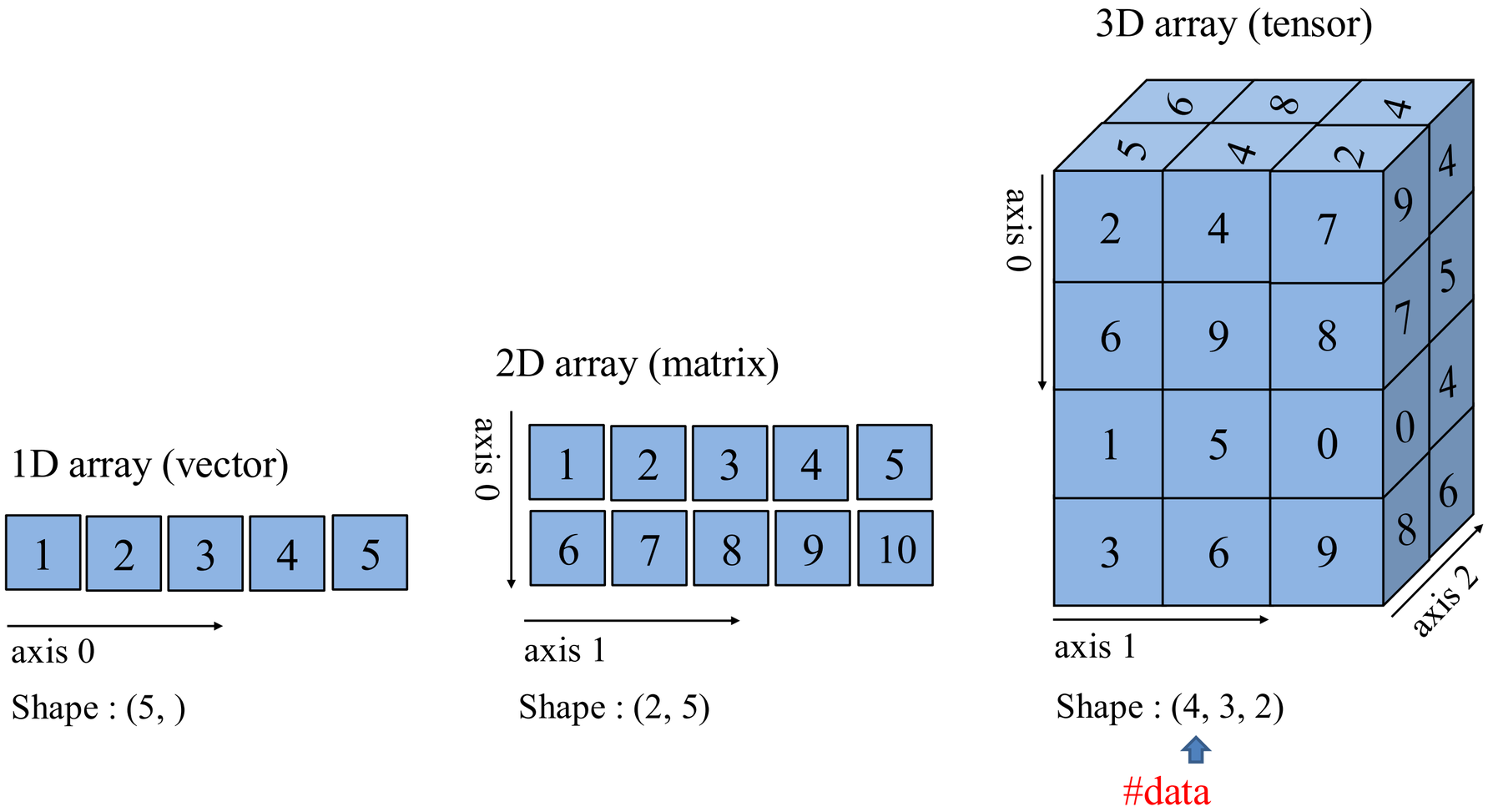
Inside: More coverage from the Grammy Awards. PAGE A8



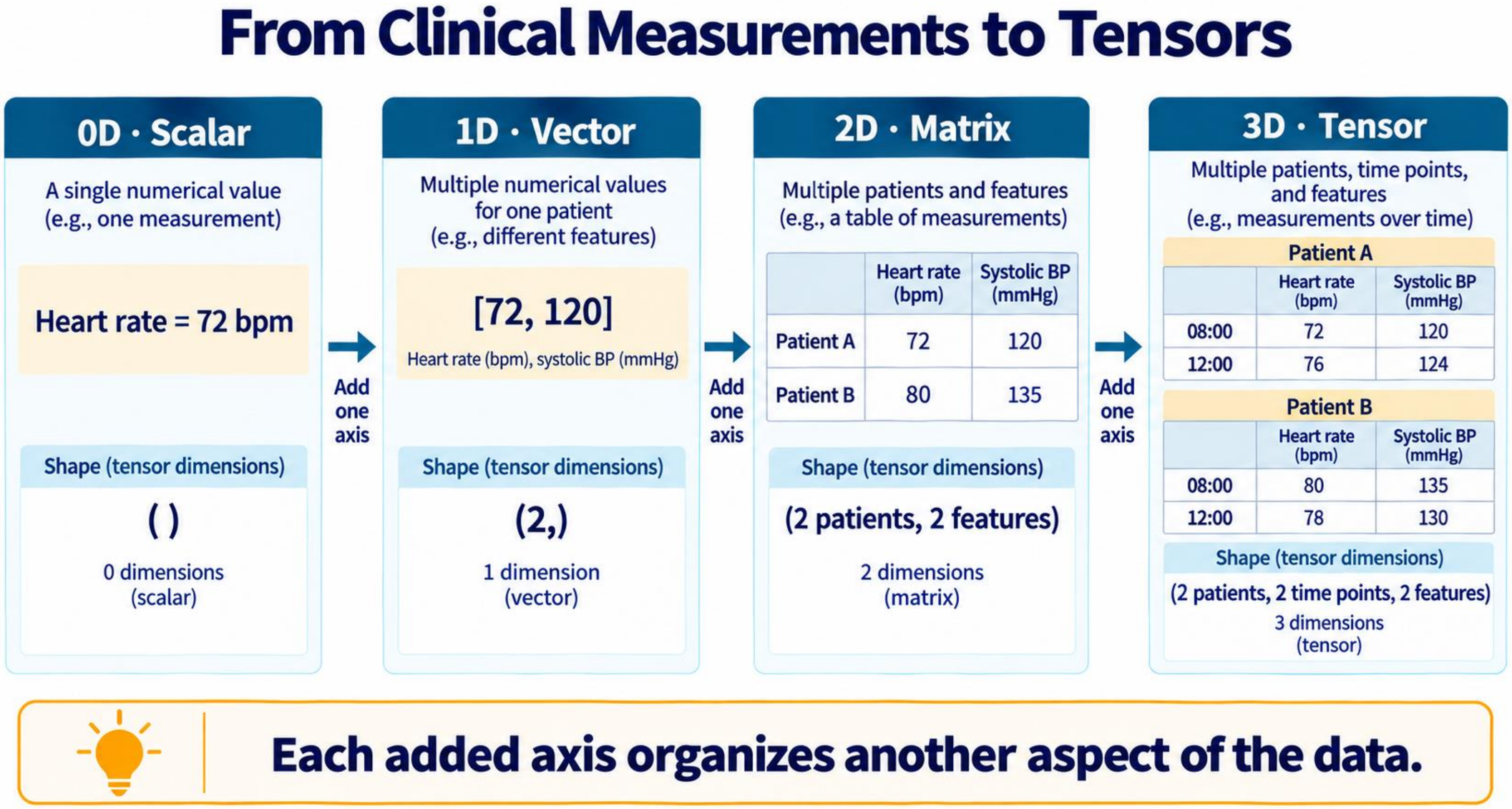
What is a Tensor? (1/3)

- When we represent data for machine learning, this generally needs to be done *numerically*. Especially when referring specifically of neural network data representation, this is accomplished via a data repository known as the *tensor*.
- A tensor is a container which can house data in *N* dimensions. Often and erroneously used interchangeably with the matrix (which is specifically a 2-dimensional tensor), tensors are generalizations of matrices to *N*-dimensional space.
- Numpy's multidimensional array ndarray is used below to create the example constructs discussed. Recall that the ndim attribute of the multidimensional array returns the number of array dimensions.

What is a Tensor? (2/3)



What is a Tensor? (3/3)



NumPy (1/2)

- What is NumPy?
  - NumPy is a Python library used for working with arrays.
  - NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.
  - NumPy stands for Numerical Python.
- Why Use NumPy?
  - In Python we have lists that serve the purpose of arrays, but they are slow to process. NumPy aims to provide an array object that is up to 50x *faster* than traditional Python lists.
  - The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy.
  - Arrays are very frequently used in data science, where speed and resources are very important.

https://www.w3schools.com/python/numpy/numpy_intro.asp

NumPy (2/2)

- Why is NumPy Faster Than Lists?
  - NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently. This behavior is called locality of reference in computer science. This is the main reason why NumPy is faster than lists. Also it is optimized to work with latest CPU architectures.
- Which Language is NumPy written in?
  - NumPy is a Python library and is written partially in Python, but most of the parts that require fast computation are written in C or C++.

```
pip install numpy
```

https://www.w3schools.com/python/numpy/numpy_intro.asp

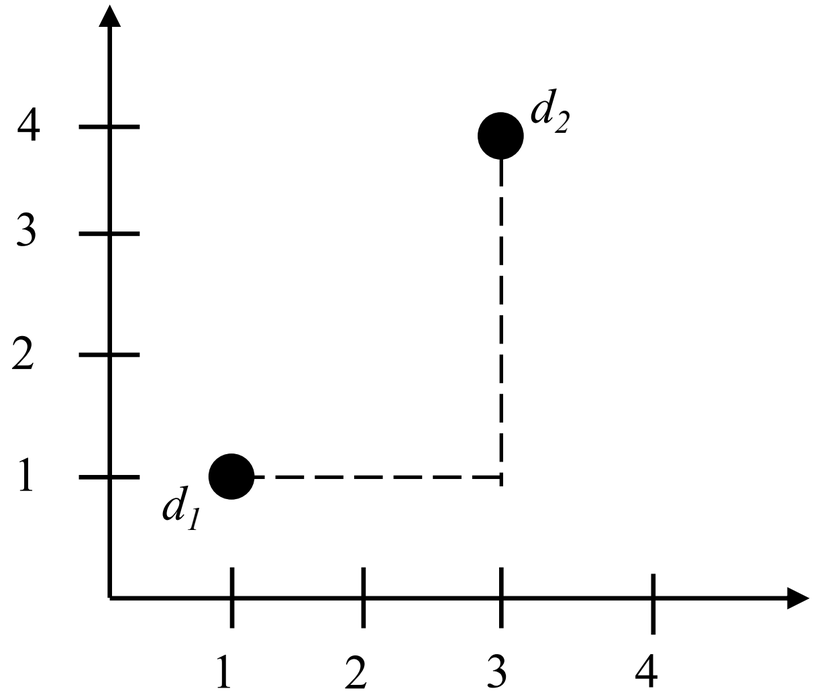
Similarity Measurement

Similarity Measurement

- A similarity measure can represent the similarity between two data, two documents, two queries, or one document and one query.
- It is possible to rank the data in the order of presumed importance.
  - Manhattan (City Block) Distance
  - Euclidean Distance
  - Cosine similarity and Cosine Distance

Manhattan (City Block) Distance

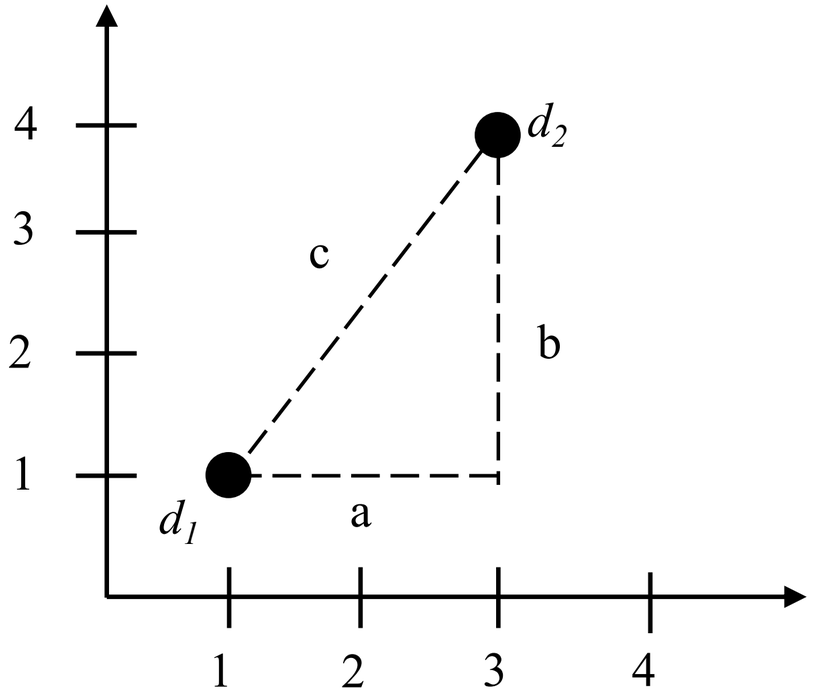
- Manhattan distance is the sum of the differences between the x- and y-coordinates.
- The Manhattan distance result is $|X_2-X_1| + |Y_2-Y_1| = |3-1| + |4-1| = 5$



Euclidean Distance

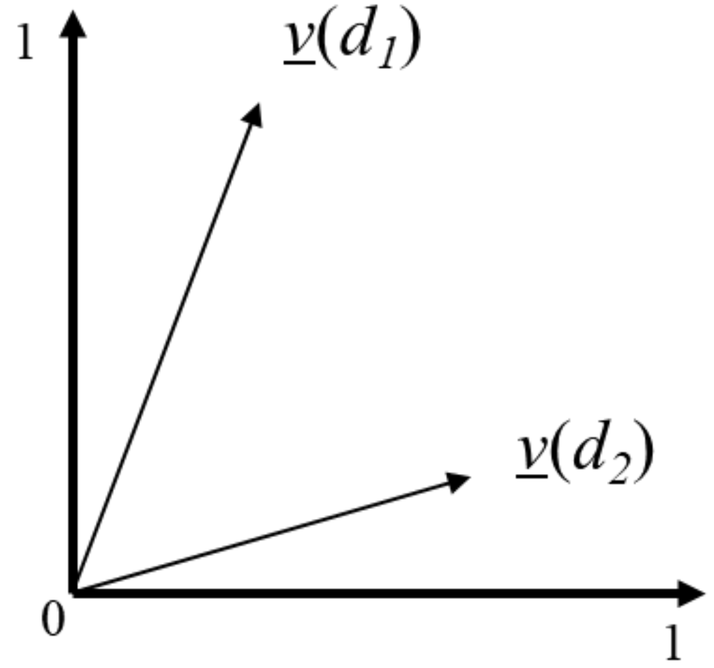
- Euclidean distance can be written as the following expression: $\sqrt{(x_2 - x_1)^2+(y_2 - y_1)^2}$
- The Euclidean distance result is $\sqrt{(3 - 1)^2+(4 - 1)^2}$

$= \sqrt{1 + 9} = 3.16$



Cosine similarity and Cosine Distance (1/4)

- We may view each data as a vector:



Cosine similarity and Cosine Distance (2/4)

– ***Cosine similarity***:

$$sim(d_1, d_2) = \frac{\underline{V}(d_1) \cdot \underline{V}(d_2)}{|\underline{V}(d_1)||\underline{V}(d_2)|}$$

inner product of vectors

$$\sum \underline{V}(d_1)_t * \underline{V}(d_2)_t$$

vector length

$$\sqrt{\sum \underline{V}(d_1)_t * \underline{V}(d_1)_t}$$

- The effect of the denominator is to normalized vectors to **unit vector**.
- The range of cosine similarity is on [0,1].
  - 1 → identical.
  - 0 → orthogonal.
- This measure is the cosine of the angle θ between the two vectors.

Cosine similarity and Cosine Distance (3/4)

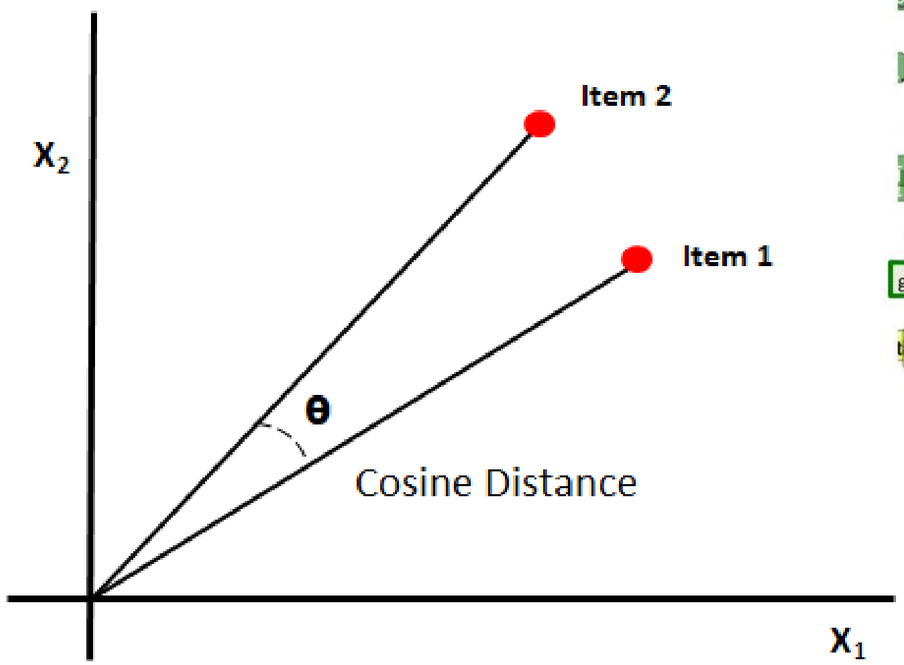


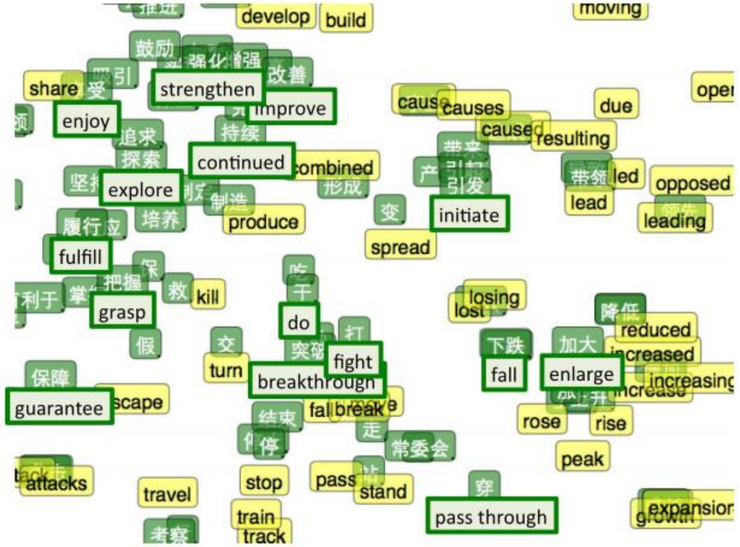
$$sim(d_1, d_2) = \frac{\underline{V}(d_1) \cdot \underline{V}(d_2)}{|\underline{V}(d_1)||\underline{V}(d_2)|}$$

$$= \frac{(0.81*0.81) + (0*3.51) + (2.02*0) + (0.56*0.56) + (0*0.36)}{\sqrt{(0.81^2 + 0 + 2.02^2 + 0.56^2 + 0)} * \sqrt{(0.81^2 + 3.51^2 + 0 + 0.56^2 + 0.36^2)}}$$

Consine Distance = 1 - cos(θ)

Cosine similarity and Cosine Distance (4/4)







Mark Zuckerberg V.S. Philip IV

https://kknews.cc/zh-tw/entertainment/kbym6p.html

How To Know Which Distance Measure To Use

- **City block distance** and **Euclidean distance** are similar because they are both concerned with the lengths of lines between two points. This fact makes them more interchangeable.
- In most cases, **Euclidean distance** will be preferable over **city block** because it is more direct in its measurement of a straight line between two points.
- **Cosine distance** is only concerned with the orientation of two points and not with their exact placement. This means that **cosine distance** is much less effected by **magnitude**, or how large your numbers are.
- **Cosine distance** is sometimes very good for text-related data. Often texts are of very different lengths. If words have vastly different counts but exist in the text in roughly the same proportion, **cosine distance** won't worry about the raw counts, only their proportional relationships to each other.

Assessing Relevance (1/2)

- To properly evaluate a system …
  - You must collect a set of test data for evaluations.
- These information needs and their relevant documents are best designed by domain experts.
  - A time-consuming and expensive process.
  - For large modern collections, it is usual to determine relevance of **only a subset of data**.
- Good practice is to collect the relevance judgments of two experts working **independently**.
  - For disagreements, a discussion between the two experts and a third senior experts will be processed.
- Seems reasonable … but … **Whether the judgments of relevance are reliable??**
- ***Kappa statistic*** – a common measure for agreement between judges, in the social sciences.



Assessing Relevance (2/2)

		Judge 2: Yes	Judge 2: No	Judge 2: Total
Judge 1	Yes	300	20	320
	No	10	70	80
	Total	310	90	400

observed agreement

expected agreement by chance

$$kappa = \frac{P(A) - P(E)}{1 - P(E)}$$

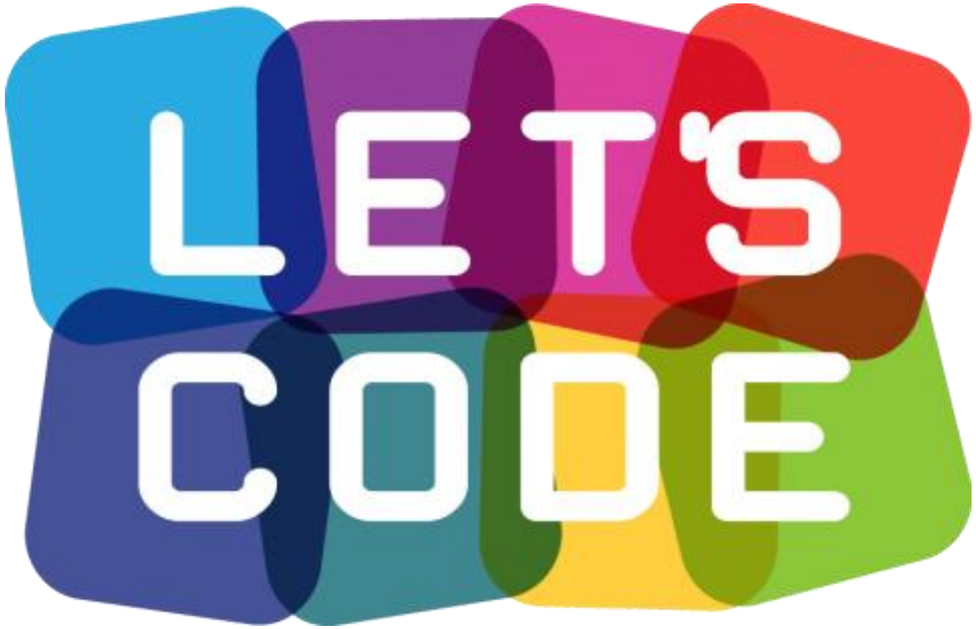
$$P(A) = (300 + 70) / 400 = 0.925$$

$$P(E) = (320/400 * 310/400) + (80/400 * 90/400) = 0.665$$

$$kappa = (0.925 - 0.665) / (1 - 0.665) = 0.776$$

- Kappa statistic lies on a [-1,1] scale.
  - 1 is perfect agreement.
    - Above 0.8 is taken as good agreement.
    - Between 0.67 and 0.8 is taken as fair agreement.
    - Below 0.67 is seen as data providing a dubious basis for an evaluation, in general.
  - 0 if they agree only at the rate given by chance.
  - Negative if they are worse than random.
    - Potential **systematic** disagreement between the judges.
- If there are more than two judges …
  - It is normal to calculate an average pairwise kappa value.

Similarity Measurement using Cosine Similarity



How to calculate cosine similarity, and how to use Sklearn for it

```python
from sklearn.metrics.pairwise import cosine_similarity

# Vectors
vec_a = [1, 2, 3, 4, 5]
vec_b = [1, 3, 5, 7, 9]

# Dot and norm
dot = sum(a*b for a, b in zip(vec_a, vec_b))
norm_a = sum(a*a for a in vec_a) ** 0.5
norm_b = sum(b*b for b in vec_b) ** 0.5

print('result:', dot / (norm_a * norm_b))
print('sklearn:', cosine_similarity([vec_a], [vec_b]))
```

```
result: 0.9972413740548081
sklearn: [[0.99724137]]
```

Thanks for your listening



Q & A

Appendix

Pandas Basic Functionality (1/5)

```
!pip install pandas
```

```
Requirement already satisfied: pandas in /opt/anaconda3/lib/python3.8/site-packages (1.1.3)
Requirement already satisfied: numpy>=1.15.4 in /opt/anaconda3/lib/python3.8/site-packages (from pandas) (1.19.5)
Requirement already satisfied: pytz>=2017.2 in /opt/anaconda3/lib/python3.8/site-packages (from pandas) (2020.1)
Requirement already satisfied: python-dateutil>=2.7.3 in /opt/anaconda3/lib/python3.8/site-packages (from pandas) (2.8.1)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.8/site-packages (from python-dateutil>=2.7.3->pandas) (1.15.0)
```

```
import pandas as pd
pd.__version__
```

```
'1.1.3'
```

```
phone = pd.Series(["Apple", "Samsung", "Mi", "Sony"])
print(phone)
```

```
0      Apple
1    Samsung
2         Mi
3       Sony
dtype: object
```

```
phone = pd.Series(["Apple", "Samsung", "Mi", "Sony"], index=["p1", "p2", "p3", "p4"])
print(phone)
```

```
p1      Apple
p2    Samsung
p3         Mi
p4       Sony
dtype: object
```

```
phone = pd.Series(["Apple", "Samsung", "Mi", "Sony"], index=["p1", "p2", "p3", "p4"])
print(phone[1])
print(phone["p3"])
```

```
Samsung
Mi
```

```
phone = pd.Series(["Apple", "Samsung", "Mi", "Sony"])
data = pd.Series(["Htc", "Oppo"])
combined = phone.append(data)
print(combined)
```

```
0      Apple
1    Samsung
2         Mi
3       Sony
0        Htc
1       Oppo
dtype: object
```

```
phone = pd.Series(["Apple", "Samsung", "Mi", "Sony"])
data = pd.Series(["Htc", "Oppo"])
combined = phone.append(data, ignore_index=True)
print(combined)
```

```
0      Apple
1    Samsung
2         Mi
3       Sony
4        Htc
5       Oppo
dtype: object
```

```
phone = pd.Series(["Apple", "Samsung", "Mi", "Sony"])
phone[1] = "Oppo"
print(phone)
```

```
0    Apple
1     Oppo
2       Mi
3     Sony
dtype: object
```

https://www.learncodewithmike.com/2020/10/python-pandas-series-tutorial.html

Pandas Basic Functionality (2/5)

```python
grades = {
    "name": ["Mike", "Sherry", "Cindy", "John"],
    "math": [80, 75, 93, 86],
    "chinese": [63, 90, 85, 70]
}

df = pd.DataFrame(grades)

print("construct df using dictionary：")
df
```

```
construct df using dictionary：
```

	name	math	chinese
0	Mike	80	63
1	Sherry	75	90
2	Cindy	93	85
3	John	86	70

```python
grades = [
    ["Mike", 80, 63],
    ["Sherry", 75, 90],
    ["Cindy", 93, 85],
    ["John", 86, 70]
]

new_df = pd.DataFrame(grades)

print("construct df using array：")
new_df
```

```
construct df using array：
```

	0	1	2
0	Mike	80	63
1	Sherry	75	90
2	Cindy	93	85
3	John	86	70

```python
df.index = ["s1", "s2", "s3", "s4"]
df.columns = ["student_name", "math_score", "chinese_score"]
df
```

	student_name	math_score	chinese_score
s1	Mike	80	63
s2	Sherry	75	90
s3	Cindy	93	85
s4	John	86	70

```python
df.tail(3)
```

	student_name	math_score	chinese_score
s2	Sherry	75	90
s3	Cindy	93	85
s4	John	86	70

```python
df.head(2)
```

	student_name	math_score	chinese_score
s1	Mike	80	63
s2	Sherry	75	90

Pandas Basic Functionality (3/5)

```
1 phone = pd.Series(["Apple", "Samsung", "Mi", "Sony"])
2 print(phone.size)
```

```
4
```

```
1 numbers = pd.Series([22, 5, 10, 12, 6, 30])
2 print(numbers.max())
3 print(numbers.min())
4 print(numbers.sum())
5 print(numbers.mean())
```

```
30
5
85
14.166666666666666
```

```
1 print(numbers.nlargest(2))
```

```
5    30
0    22
dtype: int64
```

```
1 print(numbers.nsmallest(2))
```

```
1    5
4    6
dtype: int64
```

Pandas Basic Functionality (4/5)

```
1  dic = {
2      "col 1": [1, 2, 3],
3      "col 2": [10, 20, 30],
4      "col 3": list('xyz'),
5      "col 4": ['a', 'b', 'c'],
6      "col 5": pd.Series(range(3))
7  }
8  df = pd.DataFrame(dic)
9  df
```

	col 1	col 2	col 3	col 4	col 5
0	1	10	x	a	0
1	2	20	y	b	1
2	3	30	z	c	2

Pandas Basic Functionality (5/5)

```
print("get column with data type 'Series'")
print(df["student_name"])

print("================================")

print("get column with data type 'DataFrame'")
print(df[["student_name"]])

print("================================")

print("get multiple columns with data type 'DataFrame'")
print(df[["student_name", "chinese_score"]])

print("================================")

print("get data with index 0~2")
print(df[0:3])
```

```
get column with data type 'Series'
s1      Mike
s2    Sherry
s3     Cindy
s4      John
Name: student_name, dtype: object
================================
get column with data type 'DataFrame'
   student_name
s1         Mike
s2       Sherry
s3        Cindy
s4         John
================================
get multiple columns with data type 'DataFrame'
   student_name  chinese_score
s1         Mike             63
s2       Sherry             90
s3        Cindy             85
s4         John             70
================================
get data with index 0~2
   student_name  math_score  chinese_score
s1         Mike          80             63
s2       Sherry          75             90
s3        Cindy          93             85
```

```
print("get column 'math_score' with index=1 using at()")
df.at['s3', 'math_score']
```

```
get column 'math_score' with index=1 using at()
93
```

```
print(df.iat[1, 0])
```

```
Sherry
```

```
print(df.iloc[[1, 3], [0, 2]])
```

```
     name  chinese
1  Sherry       90
3    John       70
```

https://www.learncodewithmike.com/2020/11/python-pandas-dataframe-tutorial.html

Fill column NaN with 0

```
1  df = pd.util.testing.makeMissingDataframe().head(10)
2  df
```

	A	B	C	D
opH1ml1gBd	0.125450	1.059896	-0.367540	-0.058248
V1528q3A3J	0.439919	NaN	1.202301	-0.953006
Qjb8kix34Q	-1.540726	1.116544	NaN	-1.136721
DlsS5B2kUq	NaN	0.687338	0.195719	-0.277763
MqRobyCvsa	0.225495	-0.336110	0.429086	-1.130543
yDWcGEe3e7	-1.365772	-0.263366	-1.745723	0.222251
Hp0XBb7lZR	1.002745	NaN	1.009483	0.277726
TCkIzY4x0n	-0.379151	-0.691578	-0.254381	-0.136602
qzjVOzB6M6	-0.276558	2.216009	-0.915985	-0.574702
lxsC3NjdLK	-0.288307	-0.738153	-0.381543	-1.298912

Fill column NaN with 0 (cont.)

```
1  df.fillna(0)
```

	A	B	C	D
opH1ml1gBd	0.125450	1.059896	-0.367540	-0.058248
V1528q3A3J	0.439919	0.000000	1.202301	-0.953006
Qjb8kix34Q	-1.540726	1.116544	0.000000	-1.136721
DlsS5B2kUq	0.000000	0.687338	0.195719	-0.277763
MqRobyCvsa	0.225495	-0.336110	0.429086	-1.130543
yDWcGEe3e7	-1.365772	-0.263366	-1.745723	0.222251
Hp0XBb7lZR	1.002745	0.000000	1.009483	0.277726
TCkIzY4x0n	-0.379151	-0.691578	-0.254381	-0.136602
qzjVOzB6M6	-0.276558	2.216009	-0.915985	-0.574702
lxsC3NjdLK	-0.288307	-0.738153	-0.381543	-1.298912

Delete Column and Row

```
1  df = pd.util.testing.makeDataFrame().head()
2  df
```

	A	B	C	D
Jp9pvqlEfh	-0.952085	0.182694	1.387359	0.297803
dwFqBzbQOA	0.123026	-0.619009	-0.809587	-0.402403
atePTaP02G	0.648440	0.335468	1.441304	1.426238
NO36Z53l6Q	3.095627	-1.861423	-0.803150	1.946054
uJw4qmMWja	-0.270624	-0.789484	1.263285	-0.292494

Delete Column and Row (cont.)

```
1  columns = ['B', 'D']
2  df.drop(columns, axis=1)
```

	A	C
Jp9pvqlEfh	-0.952085	1.387359
dwFqBzbQOA	0.123026	-0.809587
atePTaP02G	0.648440	1.441304
NO36Z53l6Q	3.095627	-0.803150
uJw4qmMWja	-0.270624	1.263285

```
1  df.drop('atePTaP02G')
```

	A	B	C	D
Jp9pvqlEfh	-0.952085	0.182694	1.387359	0.297803
dwFqBzbQOA	0.123026	-0.619009	-0.809587	-0.402403
NO36Z53l6Q	3.095627	-1.861423	-0.803150	1.946054
uJw4qmMWja	-0.270624	-0.789484	1.263285	-0.292494

Filter Dataset

```
df = pd.read_csv('http://bit.ly/kaggletrain')
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Filter Dataset (cont.)

```
1  df.loc[:3, 'Pclass':'Ticket']
```

	Pclass	Sex	Age	SibSp	Parch	Ticket
0	3	male	22.0	1	0	A/5 21171
1	1	female	38.0	1	0	PC 17599
2	3	female	26.0	0	0	STON/O2. 3101282
3	1	female	35.0	1	0	113803

Filter Dataset (cont.)

```
1  age = 70
2  df.query("Age > @age & Sex == 'male'")
```

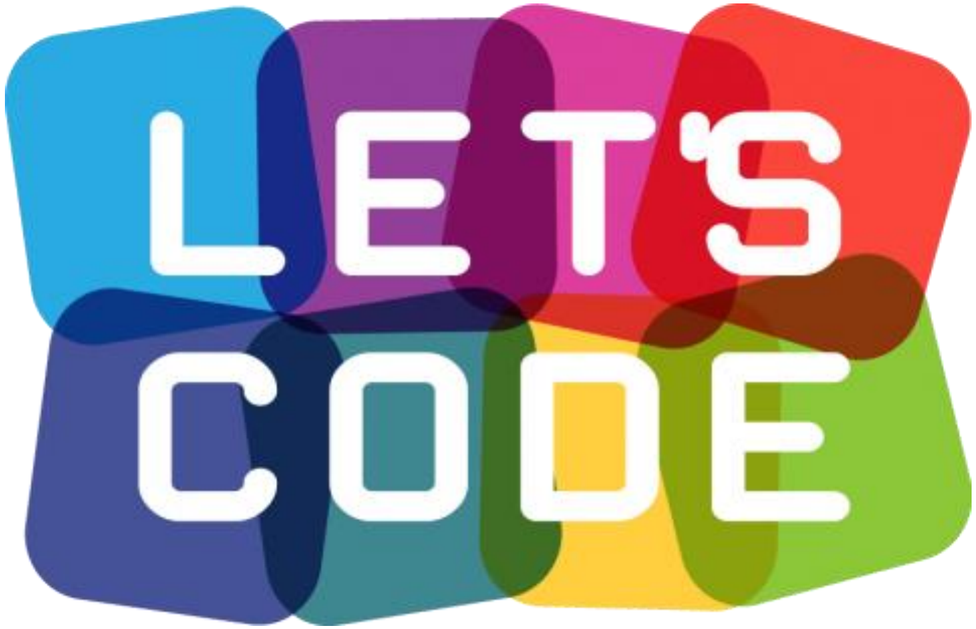
	PassengerId	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
96	97	0	1	male	71.0	0	0	PC 17754	34.6542	A5	C
116	117	0	3	male	70.5	0	0	370369	7.7500	NaN	Q
493	494	0	1	male	71.0	0	0	PC 17609	49.5042	NaN	C
630	631	1	1	male	80.0	0	0	27042	30.0000	A23	S
851	852	0	3	male	74.0	0	0	347060	7.7750	NaN	S

Filter Dataset (cont.)

```
1  df[df.isnull().any(axis=1)].head().style.highlight_null()
```

	PassengerId	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	male	22.000000	1	0	A/5 21171	7.250000	nan	S
2	3	1	3	female	26.000000	0	0	STON/O2. 3101282	7.925000	nan	S
4	5	0	3	male	35.000000	0	0	373450	8.050000	nan	S
5	6	0	3	male	nan	0	0	330877	8.458300	nan	Q
7	8	0	3	male	2.000000	3	1	349909	21.075000	nan	S

Tensor Explained with NumPy



Scalar

- A single number is what constitutes a scalar. A scalar is a 0-dimensional (0D) tensor. It, thus, has 0 axes, and is of rank 0 (tensor-speak for 'number of axes').

```
1  import numpy as np
2
3  x = np.array(5)
4  print(x)
5  print('A scalar is %dD tensor' %(x.ndim))
```

```
5
A scalar is 0D tensor
```

Vector

- A vector is a single dimension (1D) tensor, which you will more commonly hear referred to in computer science as an array. An vector is made up of a series of numbers, has 1 axis, and is of rank 1.

```
1 x = np.array([5, 6, 7, 3, 5, 4])
2 print(x)
3 print('A vector is %dD tensor' %(x.ndim))
```

```
[5 6 7 3 5 4]
A vector is 1D tensor
```

Matrix

- A matrix is arranged as a grid of numbers (think rows and columns), and is technically a 2 dimension (2D) tensor.

```
1  x = np.array([[1, 4, 7],
2                [2, 5, 8],
3                [3, 6, 9]])
4  print(x)
5  print('A matrix is %dD tensor' %(x.ndim))
```

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
A matrix is 2D tensor
```

3D Tensor (and higher dimensionality)

- While, technically, all of the above constructs are valid tensors, colloquially when we speak of tensors we are generally speaking of the generalization of the concept of a matrix to $N \geq 3$ dimensions.

```
1  x = np.array([[[1, 4, 7],
2                 [2, 5, 8],
3                 [3, 6, 9]],
4                [[10, 40, 70],
5                 [20, 50, 80],
6                 [30, 60, 90]],
7                [[100, 400, 700],
8                 [200, 500, 800],
9                 [300, 600, 900]]])
10 print(x)
11 print('This tensor is >= %dD tensor' %(x.ndim))
```

```
[[[  1   4   7]
  [  2   5   8]
  [  3   6   9]]

 [[ 10  40  70]
  [ 20  50  80]
  [ 30  60  90]]

 [[100 400 700]
  [200 500 800]
  [300 600 900]]]
This tensor is >= 3D tensor
```

reshape and ravel

```
a1 = np.arange(1, 13)
```

1	2	3	4	5	6	7	8	9	10	11	12

```
a1.ravel()
```

1	2	3	4
5	6	7	8
9	10	11	12

```
a1.reshape(3, 4)
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

```
a1.reshape(-1, 4)
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

```
a1.reshape(3, -1)
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

1	4	7	10
2	5	8	11
3	6	9	12

```
a1.reshape(3, -1, order='F')
```

```
array([[ 1,  4,  7, 10],
       [ 2,  5,  8, 11],
       [ 3,  6,  9, 12]])
```

Stack

```
a1 = np.arange(1, 13)
```

1	2	3	4	5	6	7	8	9	10	11	12

```
a2 = np.arange(13, 25)
```

13	14	15	16	17	18	19	20	21	22	23	24

```
np.stack((a1, a2))
```

```
array([[ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12],
       [13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24]])
```

1	2	3	4	5	6	7	8	9	10	11	12
13	14	15	16	17	18	19	20	21	22	23	24

```
np.hstack((a1, a2))
```

```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
       18, 19, 20, 21, 22, 23, 24])
```

1	2	3	4	5	…	…	…	21	22	23	24

```
np.stack((a1, a2), axis=1)
```

```
array([[ 1, 13],
       [ 2, 14],
       [ 3, 15],
       [ 4, 16],
       [ 5, 17],
       [ 6, 18],
       [ 7, 19],
       [ 8, 20],
       [ 9, 21],
       [10, 22],
       [11, 23],
       [12, 24]])
```

1	13
2	14
3	15
4	16
5	17
6	18
7	19
8	20
9	21
10	22
11	23
12	24

3D array from 2D arrays (1/3)

1	2	3	4
5	6	7	8
9	10	11	12

```
a1 = np.arange(1, 13).reshape(3, 4)
print(a1)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

13	14	15	16
17	18	19	20
21	22	23	24

```
a2 = np.arange(13, 25).reshape(3, -1)
print(a2)
```

```
[[13 14 15 16]
 [17 18 19 20]
 [21 22 23 24]]
```

stack along axis 0

```
a3_0 = np.stack((a1, a2))
print(a3_0)
print(a3_0.shape)
```

```
[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[13 14 15 16]
  [17 18 19 20]
  [21 22 23 24]]]
(2, 3, 4)
```



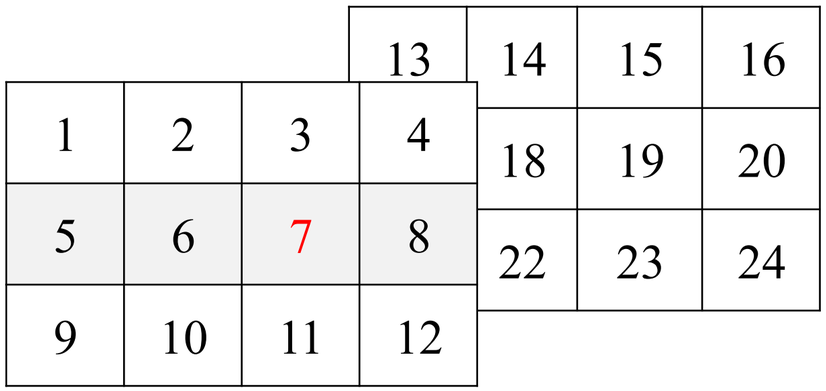
axis=0

3D array from 2D arrays (2/3)

```
a3_0
```

```
array([[[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12]],

       [[13, 14, 15, 16],
        [17, 18, 19, 20],
        [21, 22, 23, 24]]])
```




```
a3_0[0]
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

```
a3_0[0, :, :]
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

```
a3_0[0, 1]
```

```
array([5, 6, 7, 8])
```

```
a3_0[0, 1, 2]
```

```
7
```

3D array from 2D arrays (3/3)

stack along axis 1

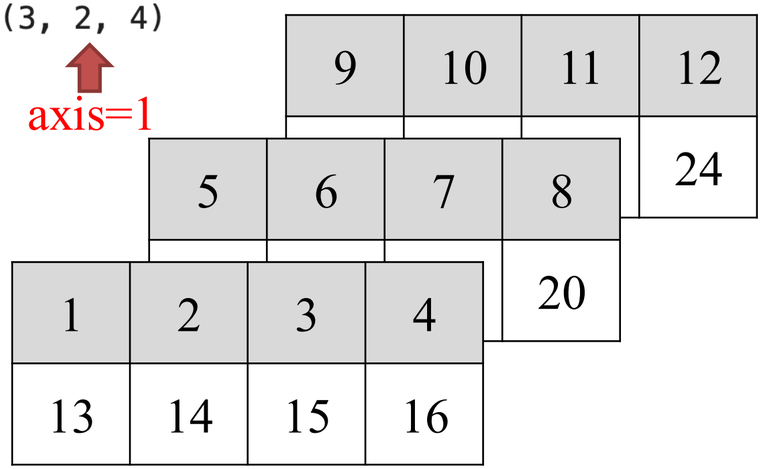
```
a3_1 = np.stack((a1, a2), axis=1)
print(a3_1)
print(a3_1.shape)
```

```
[[[ 1  2  3  4]
  [13 14 15 16]]

 [[ 5  6  7  8]
  [17 18 19 20]]

 [[ 9 10 11 12]
  [21 22 23 24]]]
(3, 2, 4)
```

axis=1



stack along axis 2

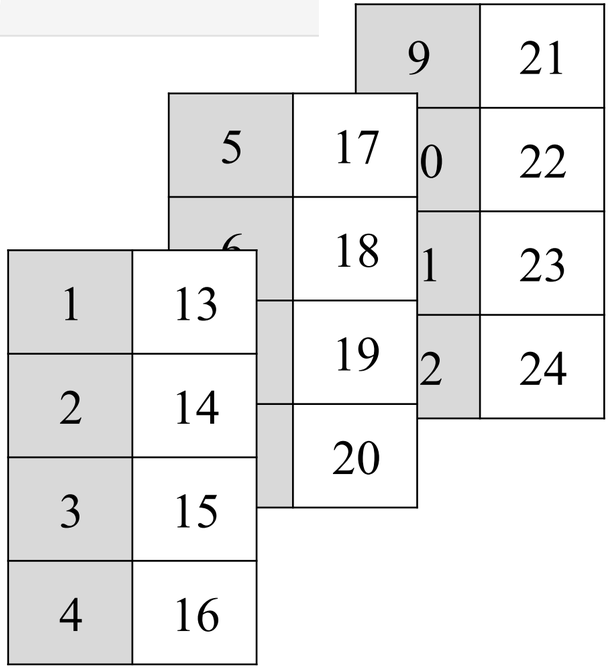
```
a3_2 = np.stack((a1, a2), axis=2)
print(a3_2)
print(a3_2.shape)
```

```
[[[ 1 13]
  [ 2 14]
  [ 3 15]
  [ 4 16]]

 [[ 5 17]
  [ 6 18]
  [ 7 19]
  [ 8 20]]

 [[ 9 21]
  [10 22]
  [11 23]
  [12 24]]]
(3, 4, 2)
```

axis=1



Reshape 3D array

```
print(a3_0)
print(a3_0.shape)
```

```
[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[13 14 15 16]
  [17 18 19 20]
  [21 22 23 24]]]
(2, 3, 4)
```

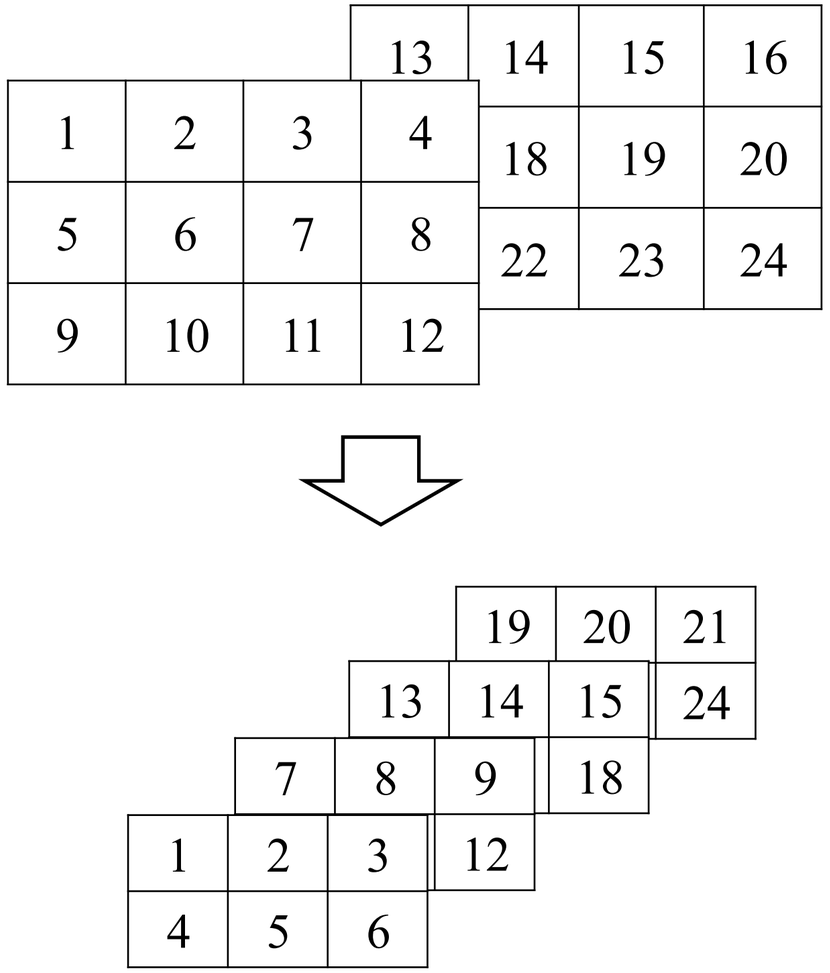
```
a3_0.reshape(4, 2, 3)
```

```
array([[[ 1,  2,  3],
        [ 4,  5,  6]],

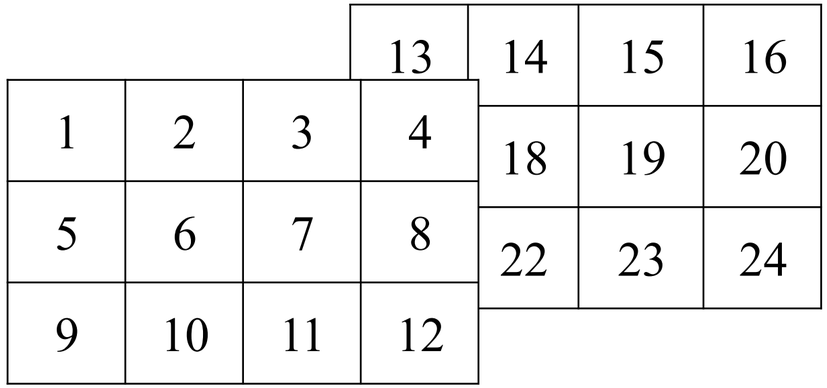
       [[ 7,  8,  9],
        [10, 11, 12]],

       [[13, 14, 15],
        [16, 17, 18]],

       [[19, 20, 21],
        [22, 23, 24]]])
```



Flatten 3D array




```
a3_0.ravel()
```

```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
       18, 19, 20, 21, 22, 23, 24])
```

```
a3_0.ravel(order='F')
```

```
array([ 1, 13,  5, 17,  9, 21,  2, 14,  6, 18, 10, 22,  3, 15,  7, 19, 11,
       23,  4, 16,  8, 20, 12, 24])
```

Slicing

```
x = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
```

```
x[1, 2]
```

```
6
```

1	2	3
4	5	6
7	8	9

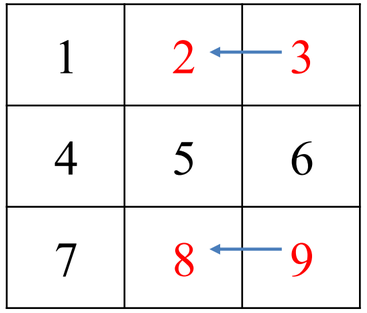
```
x[0:2, 1:3]
```

```
array([[2, 3],
       [5, 6]])
```

1	2	3
4	5	6
7	8	9

```
x[::2, 2:0:-1]
```

```
array([[3, 2],
       [9, 8]])
```




```
x[:2]
```

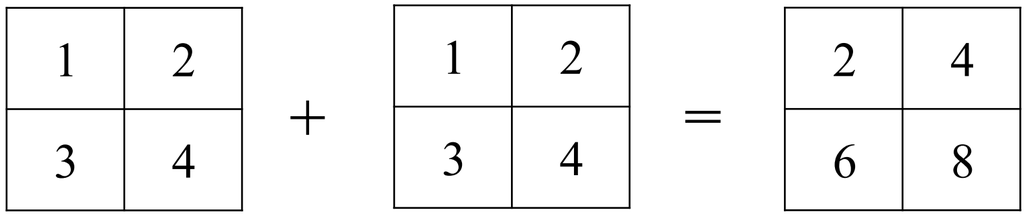
```
array([[1, 2, 3],
       [4, 5, 6]])
```

1	2	3
4	5	6
7	8	9

Element-wise

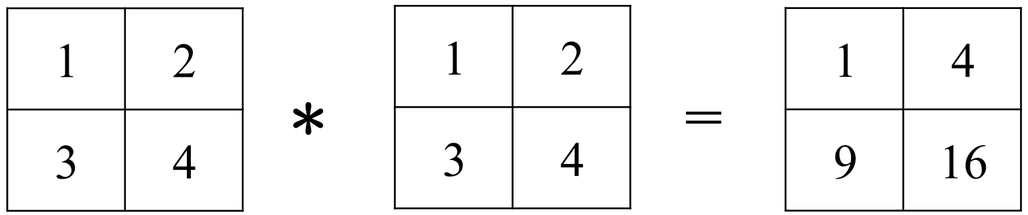
```
np.array([[1,2], [3, 4]]) + np.array([[1,2], [3, 4]])
```

```
array([[2, 4],
       [6, 8]])
```




```
np.array([[1,2], [3, 4]]) * np.array([[1,2], [3, 4]])
```

```
array([[ 1,  4],
       [ 9, 16]])
```




```
np.array([[1,2], [3, 4]]) > np.array([[1,2], [3, 4]])
```

```
array([[False, False],
       [False, False]])
```

Broadcasting

```
np.array([[1,2], [3, 4]]) + np.array([[1,2]])
```

```
array([[2, 4],
       [4, 6]])
```

```
np.array([[1], [2]]) + np.array([[3, 4]])
```

```
array([[4, 5],
       [5, 6]])
```

```
a = np.array([[[1],[2]]])
```

```
a.shape
```

```
(1, 2, 1)
```

```
b = np.array([3, 4])
```

```
b.shape
```

```
(2,)
```